Eduvos Assessment Coversheet

Academic Details

Campus: **Cape Town- Mowbray**

Faculty: **Information Technology**

Module Code: **ITMDA0-33 (2026)**

Lecturer's Name: **Marlene Morgan-Chisholm**

Group (if applicable)

Assessment Details

Assessment Type: **Project**

Assignment/Brief Number: **ITMDA0-33 Project**

Due Date/ Submission Date: **29 August 2026**

Student Details (Group, if applicable)

#	Full Name	Student Number	Contact Number	% Participation	Signature
1					
2					
3					
4					
5					

6					

Student Details (Individual, if applicable)

Student Full Name: **Kgothatso Masha**

Student Number: **EDUV4976323**

Academic Integrity

AI Disclosure Completed: Yes **Yes** ☐ **No**

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature 

Date **26 August 2026**

Lecturer Use Only

Lecturer Comments:

Marks Awarded (%):

Lecturer Signature:

Date:

Assessment

Table of Contents

Eduvos Assessment Coversheet 1

Assessment 5

SECTION A: System Requirements 9

SECTION B: Program Design 10

- Notes: 10

System Specifications: 11

1. The ER Diagram 11
2. Database and Table Creation 12
3. Insert Sample Data 12
4. Creating Views 13
5. Creating Stored Procedures 13
6. Creating Triggers 14
7. Indexes Creation 14
8. Database Backup Creation 15

SECTION C: User Documentation 16

1_Database_Table.sql 17

- Notes: 17

0. Creating The Database 17
1. Creating STUDENT Table 18
2. Creating CAREER ADVISOR Table 18
3. Creating CAREER PROGRAMME Table 19
4. Creating PROGRAMME ENROLMENT Table 19
5. Creating LEARNING ACTIVITY Table 20
6. Creating LEARNING ACTIVITY PROGRESS Table 20
7. Creating ASSESSMENT Table 21
8. Creating ASSESSMENT RESULT Table 21
9. Creating DIGITAL CAREER RESOURCE Table 22
10. Creating STUDENT RESOURCE Table 22
11. Creating ERROR EVENT LOG Table 23

2_Sample_Data_Insertion.sql 24

- Notes 24

1. Inserting into STUDENT Table 24
2. Inserting into CAREER ADVISOR Table 24
3. Inserting into CAREER PROGRAMME Table 25
4. Inserting into PROGRAMME ENROLMENT Table 25
5. Inserting into LEARNING ACTIVITY Table 26
6. Inserting into LEARNING ACTIVITY PROGRESS Table 26
7. Inserting into ASSESSMENT Table 27

8. Inserting into ASSESSMENT RESULT Table....27
9. Inserting into CAREER RESOURCE Table....28
10. Inserting into STUDENT RESOURCE....28
11. Inserting into ERROR EVENT LOG Table....29
3_Views_Creation.sql....30
- Notes:....30
1. Creating View vwBlockedStudents....30
2. Creating View vwTopCareerReadyStudents....31
3. Creating View vwMostDownloadedResources....31
4. Creating View vwPopularCareerProgrammes....32
4_Stored_Procedures_Creation.sql....33
- Notes....33
0. Using EdulaunchDB and Setting Delimiter....33
1. Creating Stored Procedure spRegisterStudent....34
2. Creating Stored Procedure spRenewAccess....35
3. Creating Stored Procedure spEnrolStudent....36
4. Creating Stored Procedure spAddCareerResource....37
5. Creating Stored Procedure spSendAccessNotice....38
5_Trigger_Creating.sql....39
- Notes....39
1. Creating Trigger trg_BFL_AutoBlock_Student....39
2. Creating Trigger trg_BFU_AutoBlock_Student....40
3. Creating Trigger trg_AFI_LogError_StudentResource....41
6_Indexes_Creation.sql....43
- Notes....43
1. Creating Index idx_Student_AccountStatus_AccessExpDate....43
2. Creating Index idx_CareerProgramme_Title_Category....43
3. Creating Index idx_Student_ProgrammeEnrolment....44
SECTION D: Rough Work....45
SECTION D: Refences....47
AI Evidence Section....48
AI Disclosure Appendix....49
Declaration:....49

SECTION A: System Requirements

- This section has the requirements for running the scripts and what I used to get the results I got when I executed the codes
- This program was run and tested on

  Socket: MySQL

  Version: 9.7.1 (MySQL Community Server – GLP

  Compiled For: Win 64 (x86_64
- Run using InnoDB engine since it allows foreign keys.

SECTION B: Program Design

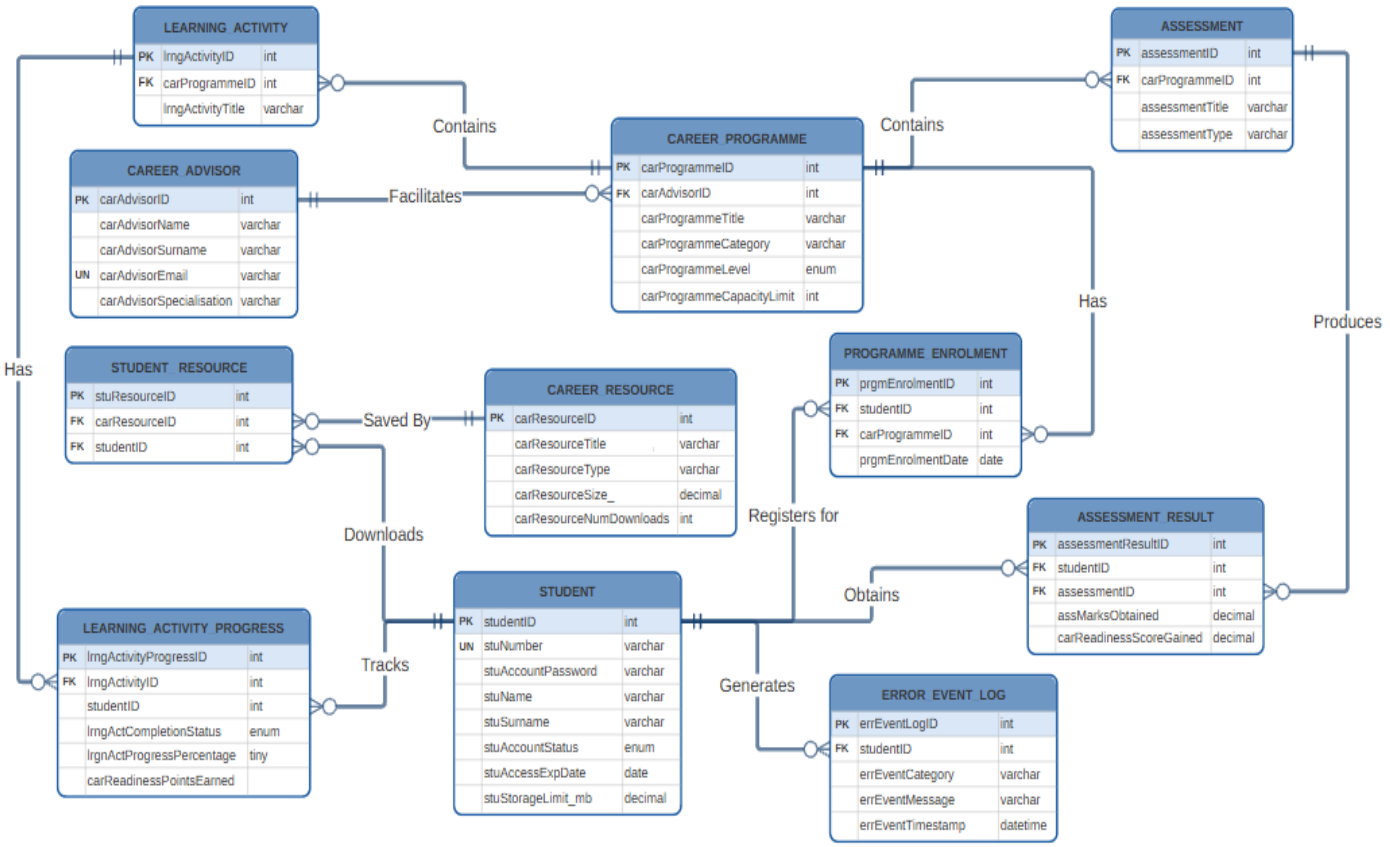
- This section contains all the screenshots that answer the key specification on the project brief?

- **Notes:**
  - Attribute: A characteristic that describes an entity.
  - Entity: A real-world object that you want to store Data about.
  - ERD: Entity Relationship Diagram is a diagram showing how different entities relate to one another
  - Primary Key (PK): is a unique key that identifies a record in the database table
  - Foreign Key (FK): is a key that comes from another table's primary key. It is used to link the tables together.
  - An instance of an entity is one specific real-world example of that entity. It is therefore a specific occurrence of something.
  - In database terms, an attribute or set of attributes that uniquely identifies the instance of the entity is needed. These unique attributes are called Unique Identifiers (UIDs). A UID is usually named the ID attribute in the entity.
  - Intersection entities are used to resolve many-to-many relationships.
  - MySQL is an SQL Relational Database Management System (RDBMS). In other words, it is a computer system that manages relational databases that reside on a database server.
  - One-to-many relationship: This relationship applies if, for each row in one table, there are many related rows in the other table.
  - One-to-one relationship: This relationship is used if, for each row in one table, there exists at most one related row in the other table
  - Many-to-many relationship: this relationship is used if, for each row in one table, there are many related rows in the other table.

    This relationship cannot be directly coded in relational database programs. It is broken down into two one-to-many relationships. A third table, sometimes referred to as a bridging or detail table, is created to keep track of the references between the two main tables.

System Specifications:

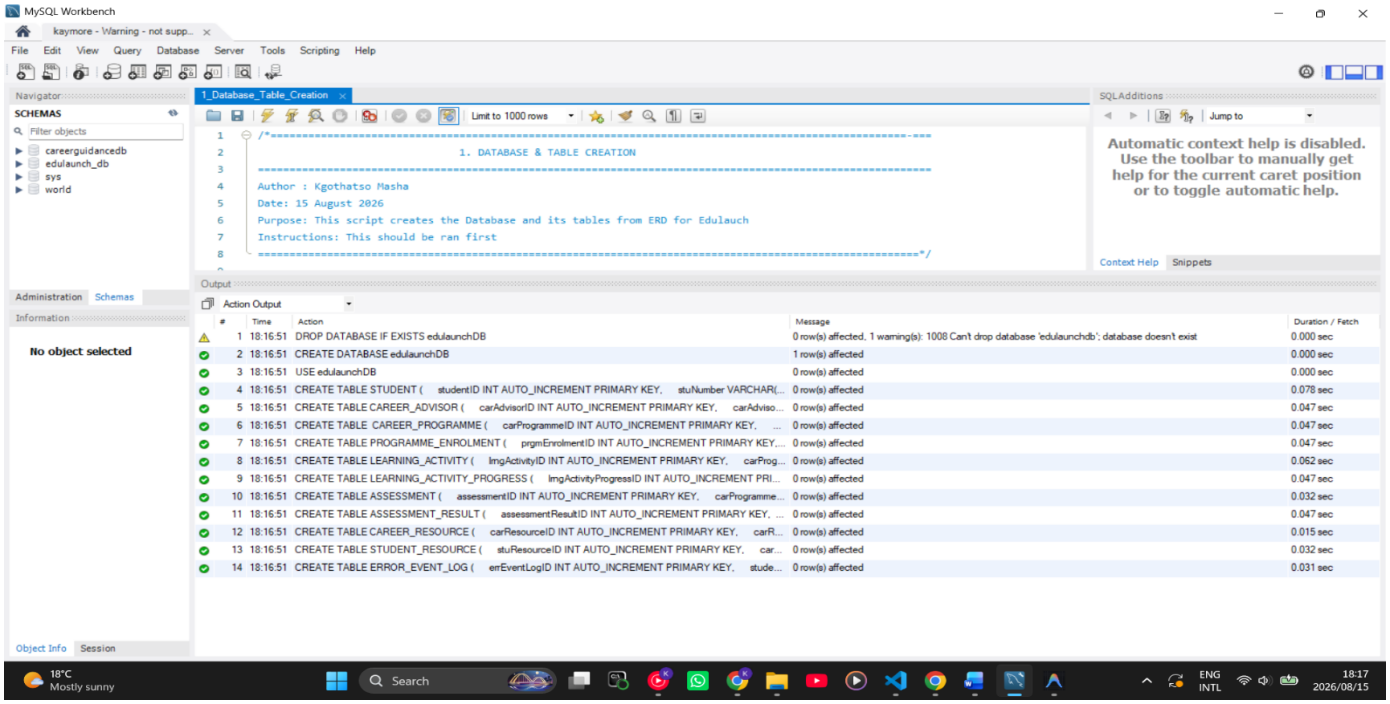
1. The ER Diagram



- **Notes**:
  - The symbol || means only one, the 0E (crow's foot) means zero or many. – e.g only one career program can be contained by zero or many assessments.
  - A career advisor facilitates a career programme which contains learning activities and assessments.
  - Car is short for Career, lrng is short for learning, stu is short for student, act is short for activity, ass is short for assessment, err is short for error, num is short for number, exp is short for expiry, prgm is short for programme,
  - I chose to use title instead of name because on the project guide on 3. Career Development Programmes they used Programme title, so I wanted that to be consistent throughout, e.g. carProgrammeTitle instead of carProgrammeName
  - Each word in the entity names is separated by a _, although it is not visible, e.g. CAREER_ADVISOR

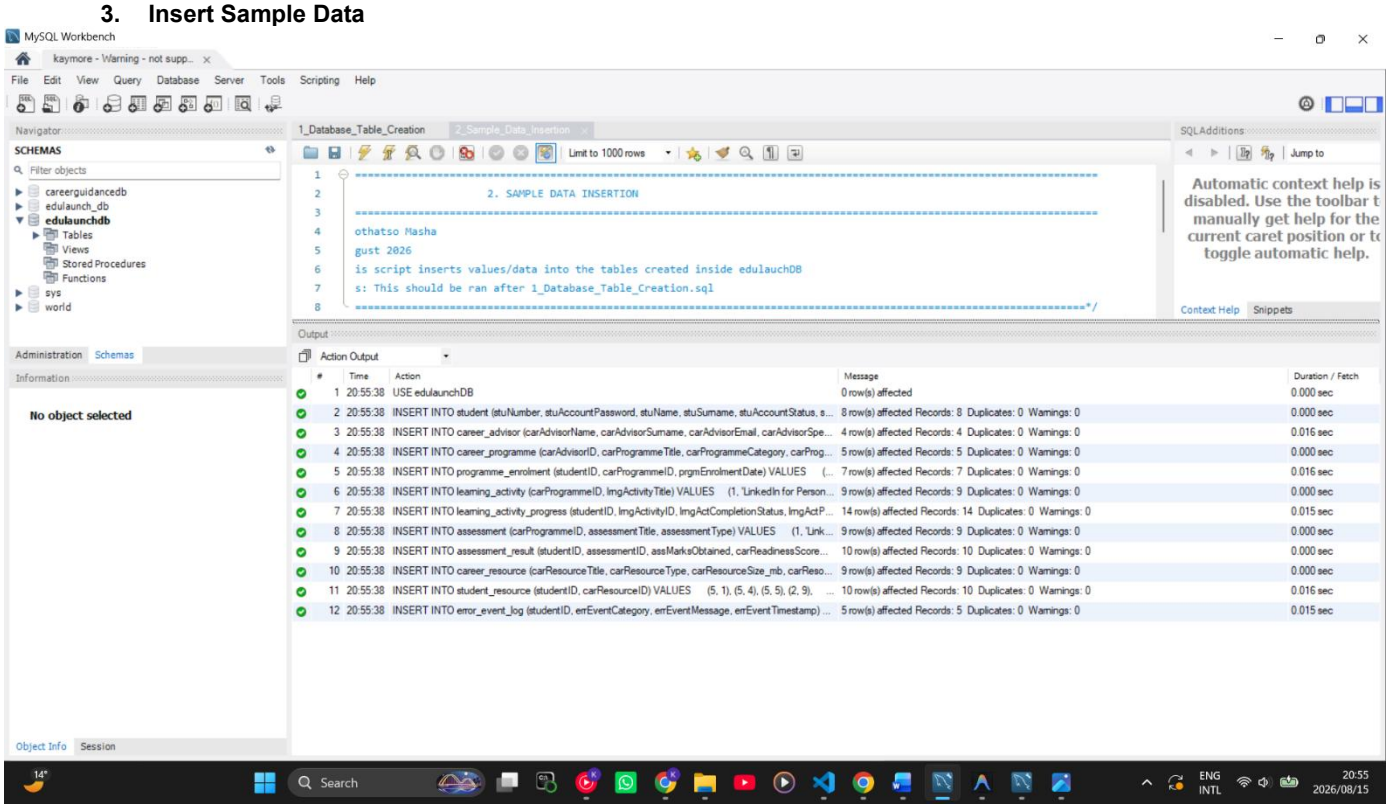
- Many-to-Many relationship resolution:
  - I created the entity PROGRAMME_ENROLMENT to solve the many-to-many relationship between STUDENT entity and CAREER_PROGRAMME entity whereby a student could have been in many career programmes, and a career programmes could have had many students, so programme enrolment acts as a linking table whereby each student's enrolment in each career programme is stored in its own row, which allows a student to have many enrolment records, one per career programme.
  - I created the entity LEARNING_ACTIVITY_PROGRESS, to solve the many-to-many relationship between STUDENT entity and LEARNING ACTIVITY whereby a student could have had many learning activities and learning activities could have been completed by many students, so learning activity progress acts as a linking table whereby each student's progress on each activity can be stored as its own row, holding that student's completion status, progress percentage, and career readiness points earned for that specific activity.
  - I created the ASSESSMENT_RESULT entity to solve the many-to-many relationship between STUDENT entity and ASSESSMENT entity whereby a student could have taken many assessments, and an assessment could have been taken by many students, so assessment acts as a linking table whereby each student's result for each assessment can be stored on its own row, holding marks obtained and career readiness score gained for that student-assessment pairing.
  - I created STUDENT_RESOURCE entity to solve the many-to-many relationship between CAREER_RESOURCE and STUDENT whereby a student could have downloaded many digital career resources, and a digital career resource could have been downloaded by many students, so student resource acts as a linking table whereby each downloaded digital career resource can be stored on its own row by student resource.

2. Database and Table Creation



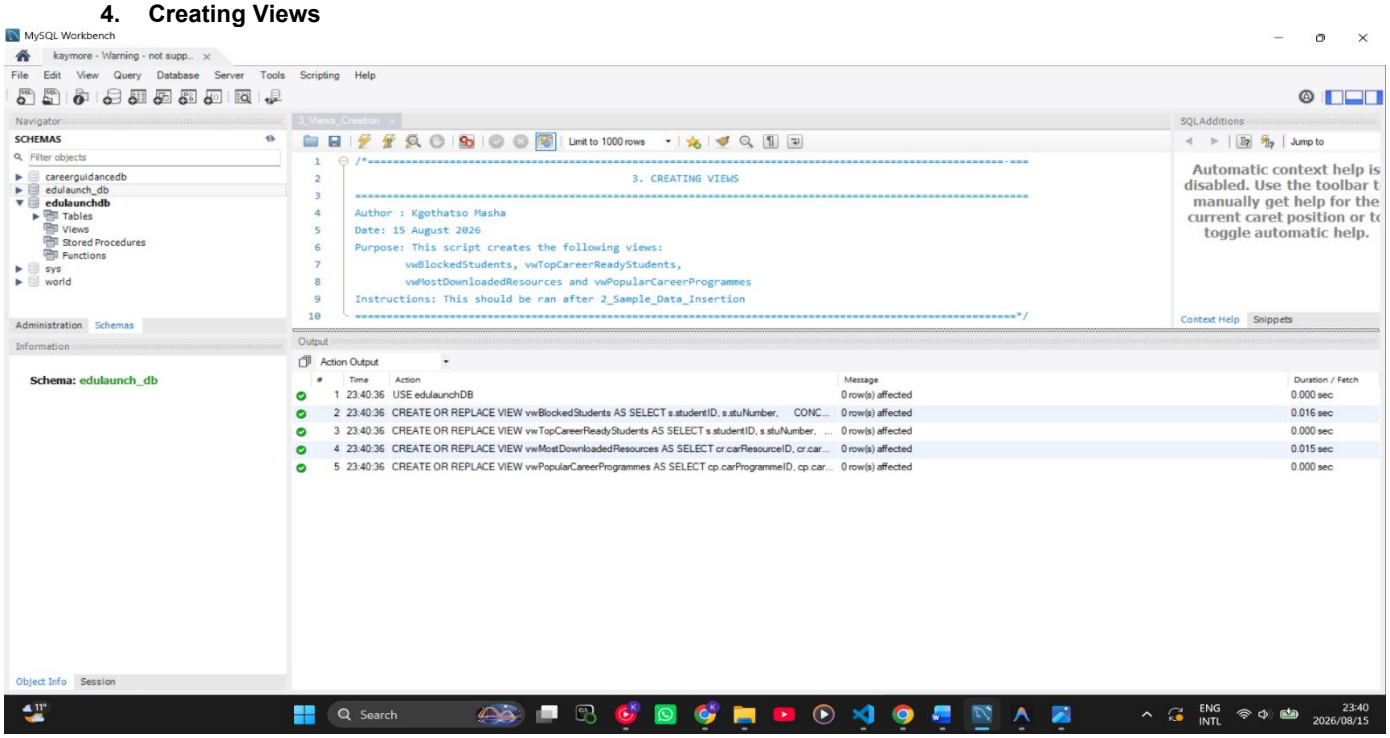
- The code for this is 1_Database_Table_Creation.sql, run this script
- This will create the required database and tables
- As you can see from the above picture everything runs successfully.
- More details about this script will be found in the user documentation section.

3. Insert Sample Data



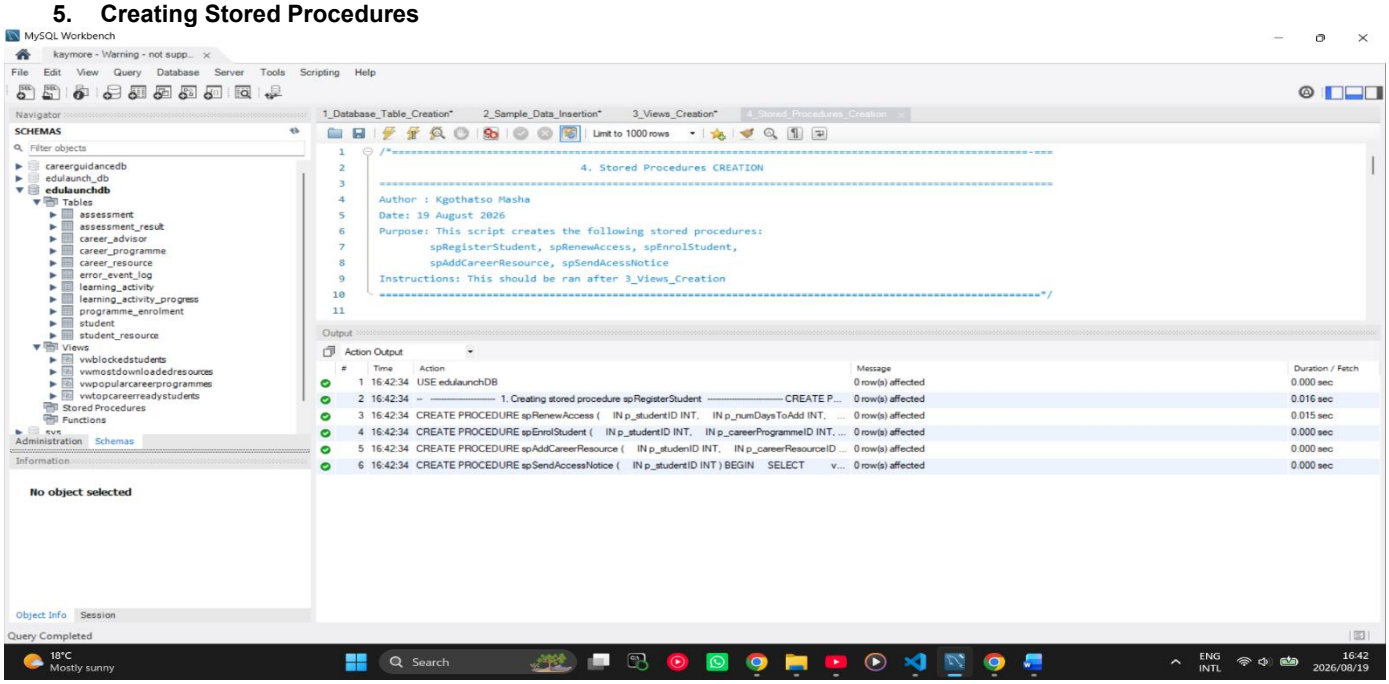
- This is what I got after running 2_Sample_Data_Insertion.sql script
- This will insert values into the edulaunchDB tables we created in the first script.
- This should be run after 1_Database_Table_Creation.sql script

4. Creating Views



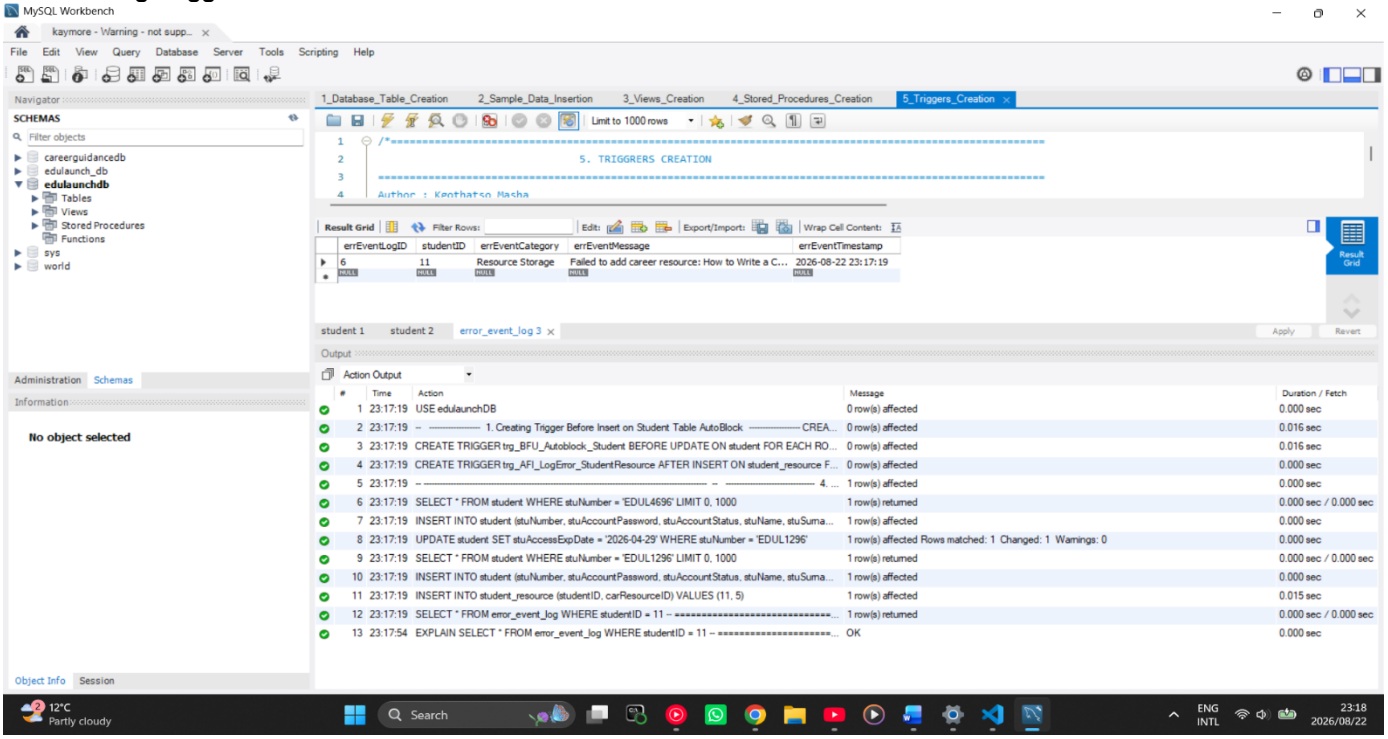
- This is what I got after running 3_Views_Creation.sql
- This script will create the required views.
- This should be run after 2_Sample_Data_Insertion.sql script

5. Creating Stored Procedures



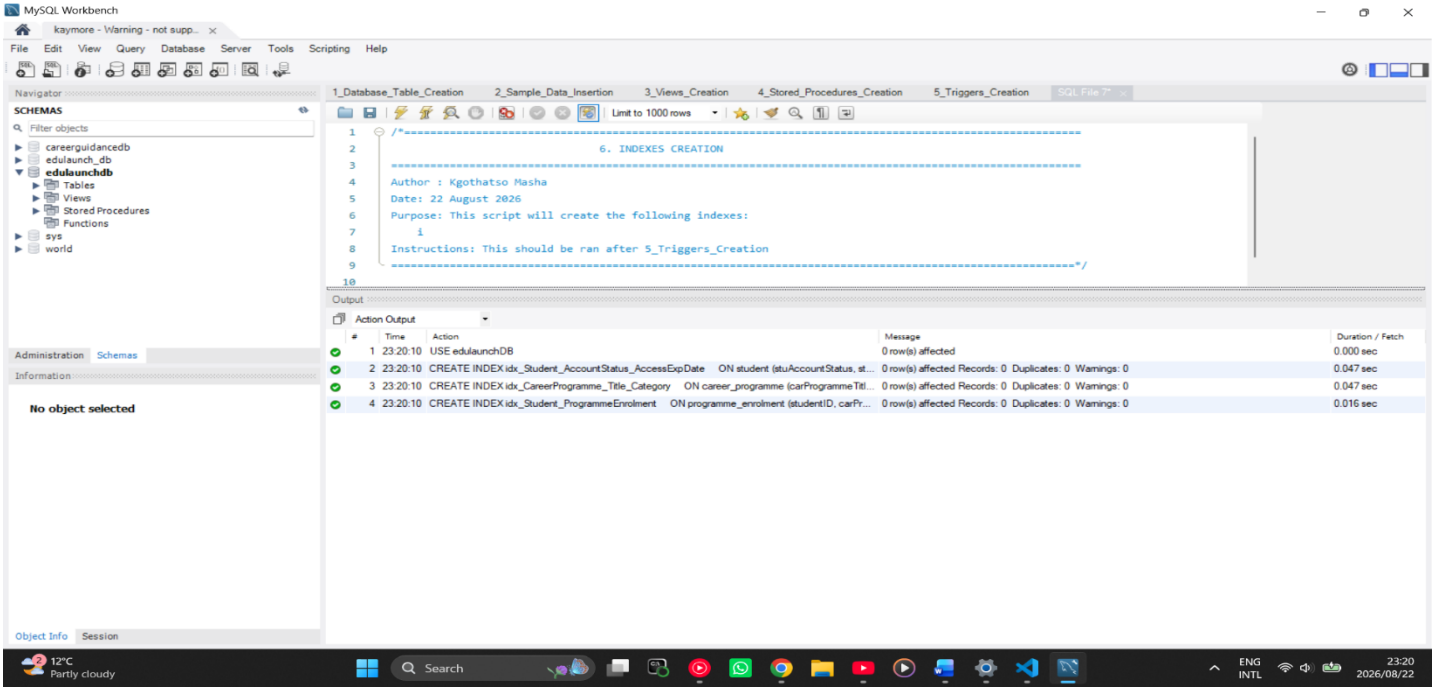
- This is what I got after running 4_Stored_Procedures_Creation.sql script.
- This script will create the required stored procedures.
- This should be run after 3_Views_Creation.sql script.

6. Creating Triggers



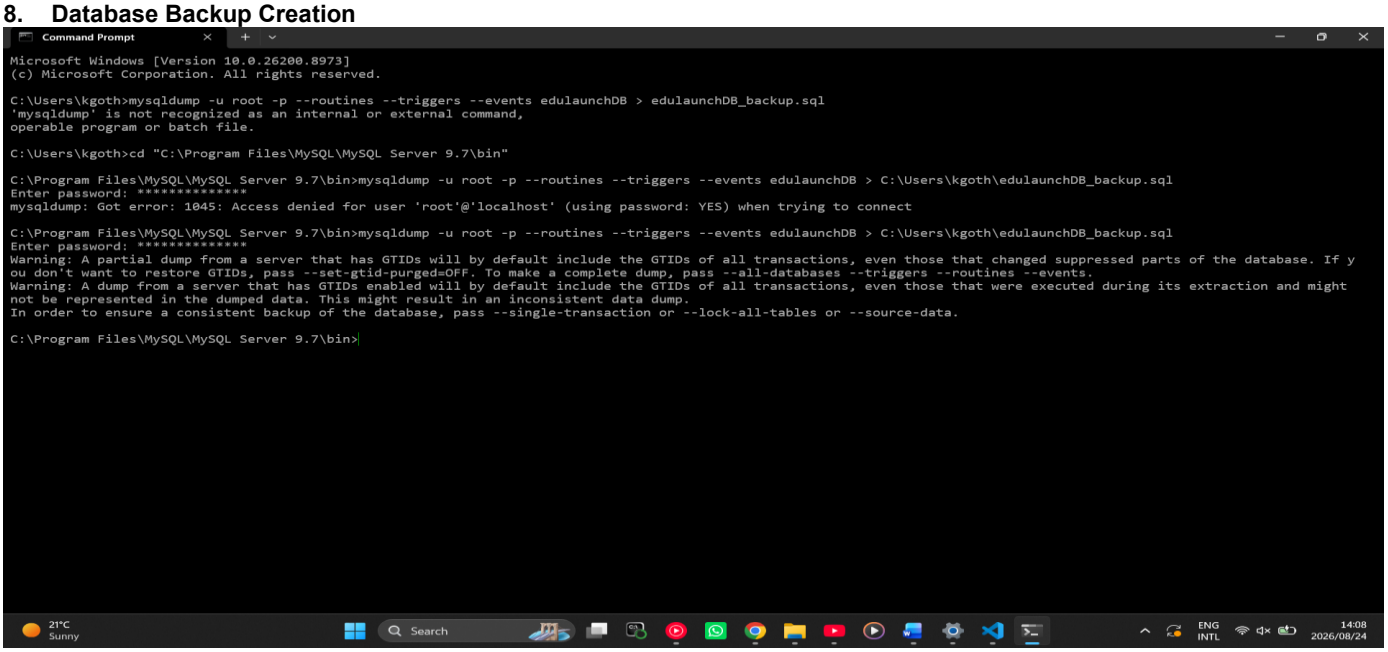
- This is what I got after running 5_Tiggers_Creation.sql script.
- This will script will create the required triggers.
- This should be run after 4_Stored_Procedures_Creation.sql script.

7. Indexes Creation

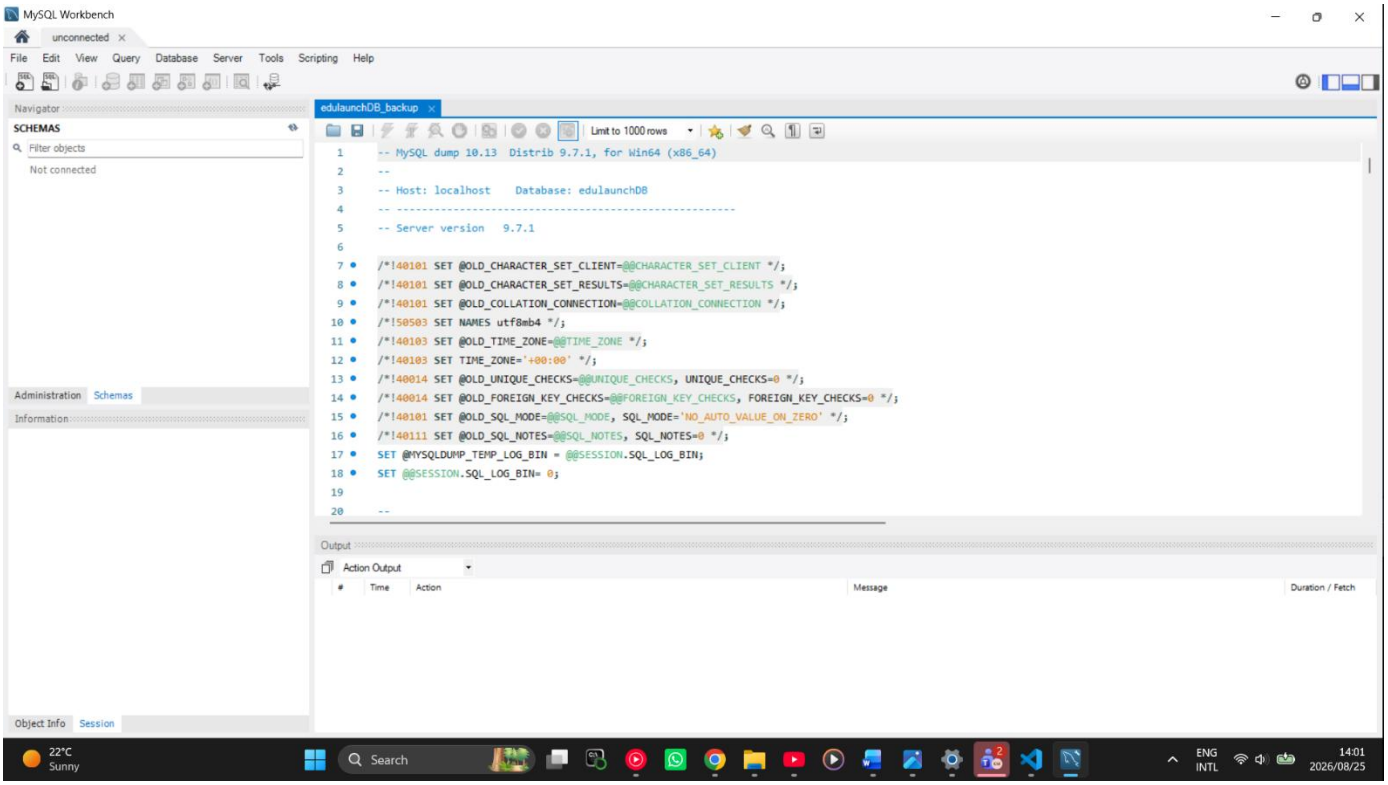


- This is what I got after running 6_Indexes_Creation.sql script.
- This script will create the required indexes.
- This should be run after 5_Stored_Procedures_Creation.

8. Database Backup Creation



This shows how I created the backup.



This shows that the backup is created.

SECTION C: User Documentation

- This section contains all the explanations of how the code works
- It also contains screenshots of successful code execution.

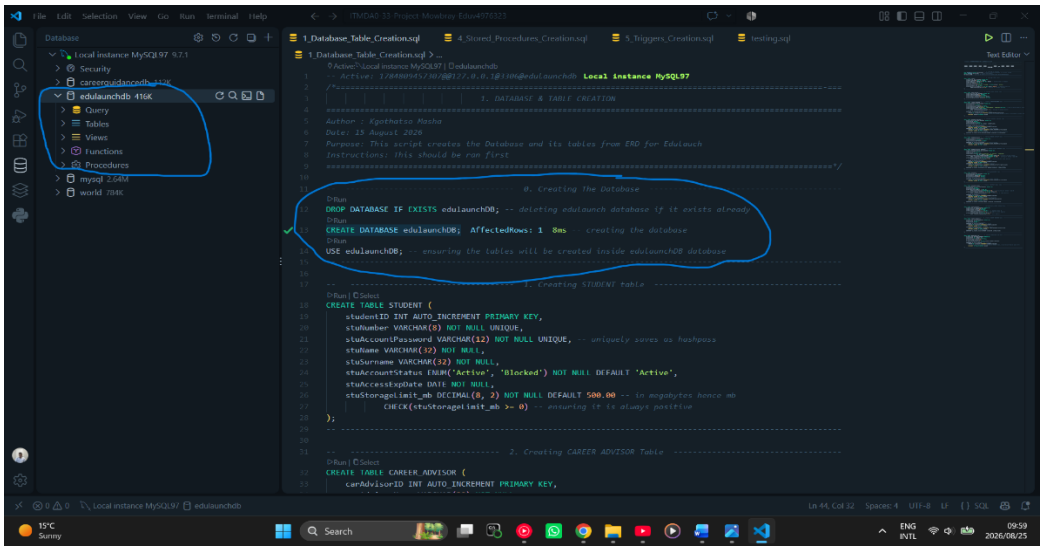
1_Database_Table.sql

- **Notes:**
  - This should be run first, it creates the database and all of its tables.
  - The names here are not exactly as they are in the code or on the ERD but are closely correlated, e.g. I might have carAdvisorID in the code but call it career advisor ID in this document.
  - The code will say CREATE TABLE X as the name, but the table will automatically be named x by MySQL, e.g. CREATE TABLE STUDENT will result in the table being named student.
  - Some of the things (especially entity names, attribute names, datatypes, etc.) are explained in more detail in section b, program design, and system specification.

- Foreign key specifications:
  - Foreign keys on MySQL only work with the InnoDB engine.
  - Whenever there is a foreign key, a constraint must be added that tells the MySQL engine that a certain value is a foreign key, meaning that value should be taken from another table, linking the two tables together, and the constraint will prevent anything that wants to destroy that link.
  - CONSTRAINT fk_x defines that foreign key constraint, using a condition like FOREIGN KEY (name of the foreign key) REFERENCES z (name of the foreign key), with z being the table the foreign key comes from, and x being whatever name is chosen for the constraint, e.g. CONSTRAINT fk_programme_advisor FOREIGN KEY (carAdvisorID) REFERENCES CAREER_ADVISOR (carAdvisorID).
  - ON DELETE RESTRICT ON UPDATE CASCADE makes sure the foreign key cannot be deleted (hence RESTRICT) on the parent table if it is still referenced on the child table, and when it is updated (hence UPDATE) on the parent table, the change will also automatically happen on the child table (hence CASCADE).
  - ON DELETE CASCADE ON UPDATE CASCADE makes sure that when the foreign key is deleted (hence DELETE) or updated (hence UPDATE) on the parent table, the change or deletion will automatically also happen on the child table (hence CASCADE).
  - ON DELETE SET NULL ON UPDATE CASCADE makes sure that when the foreign key is deleted (hence DELETE) on the parent table, it will be set to null on the child table (hence SET NULL), and when it is updated on the parent table (hence UPDATE), the change will also automatically happen here.

0. Creating The Database

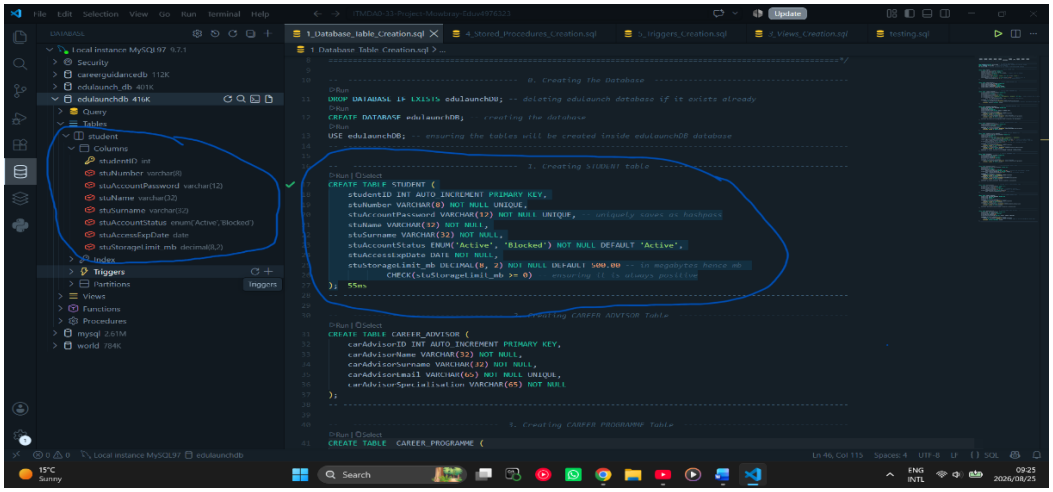
- This deletes the database first if it already exists in the system, using drop database if exists.
- Then it creates a new database called edulaunchDB, which is where all the tables, views, procedures and triggers will live, using create database.
- Use edulaunchDB makes sure that everything that comes after is created inside the database that was just created.



---

1. Creating STUDENT Table

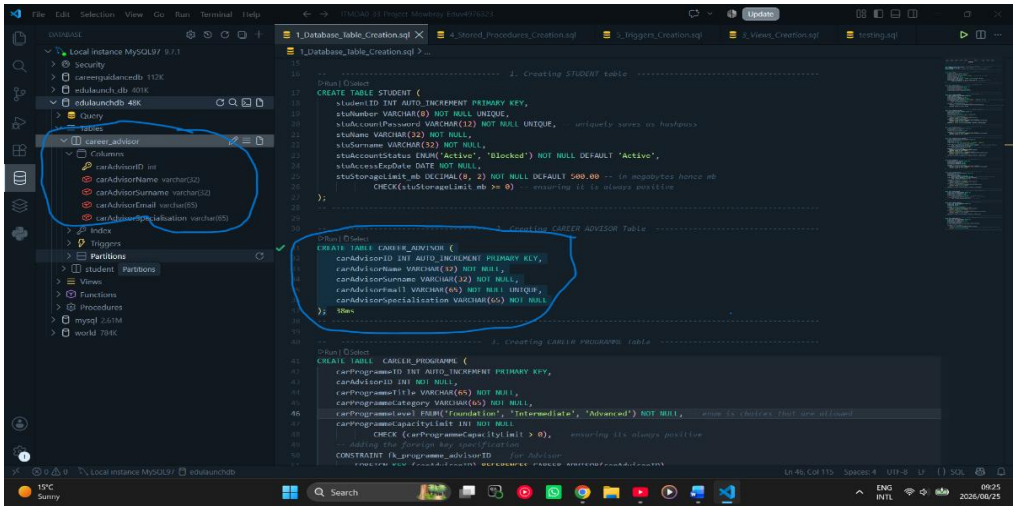
- This table stores the students' details.
- This code block creates the table (entity) STUDENT from the ERD with the following columns (attributes): student ID (the primary key, auto generated by MySQL), student number, student account password, student name, student surname, student account status (can either be Active or Blocked, with Active being the default), student access expiry date, and lastly student storage limit in megabytes (hence mb).
- A check constraint is used on stuStorageLimit_mb to make sure negative values cannot be inserted, and the default value is 500.00 megabytes.
- On student password, the password should be stored as hashpassxxxx, with x being unique numbers. hashpass is used because a student's real password should never be stored as plain text in a database as a security measure, so hashpass stands in for where the properly hashed password would sit. This should be unique to every student.
- All the columns (attributes) cannot be null (contain no value).
- stuNumber should be unique for every student, and the standard is EDULXXXX, with X being the unique student number, e.g. EDUL4698.
- I shortened student to stu because it is short, and it still tells you where the column (attribute) is from.
- This screenshot shows that the student table has been created successfully:



---

2. Creating CAREER ADVISOR Table

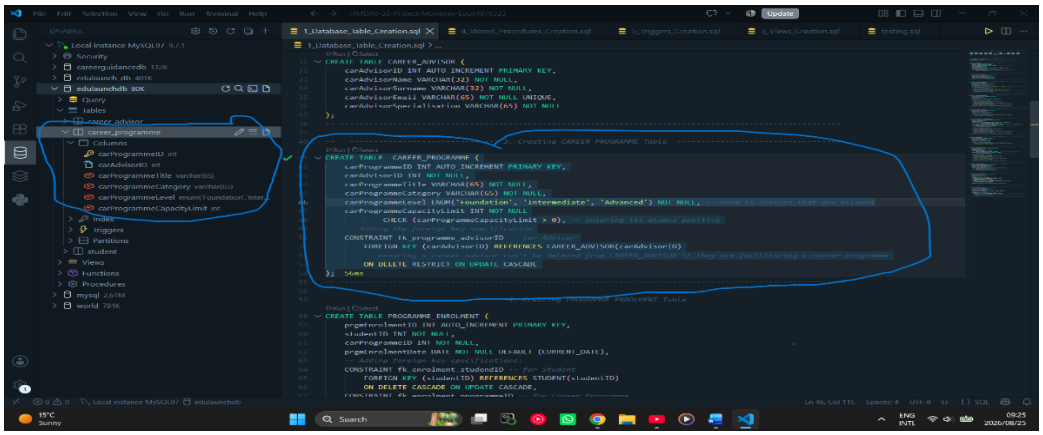
- This table stores the career advisors' details.
- This code block creates the table (entity) CAREER_ADVISOR from the ERD with the following columns (attributes): career advisor ID (the primary key, auto generated), career advisor name, career advisor surname, career advisor email address (unique to every advisor), and career advisor specialisation (things like CV support, interview preparation, workplace readiness, etc.).
- All the columns (attributes) cannot be null.
- car is short for career.
- This screen show shows that career_advisor table has been created successfully.



---

3. Creating CAREER PROGRAMME Table

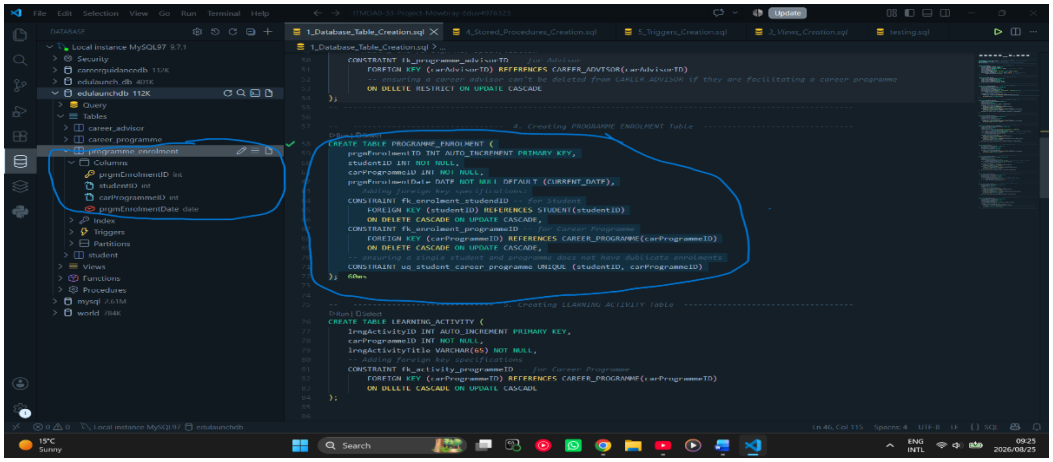
- This table stores career programme details.
- This code block creates the table (entity) CAREER_PROGRAMME from the ERD with the following columns (attributes): career programme ID (the primary key, auto generated), career advisor ID (a foreign key from the career advisor table), career programme title, career programme category (e.g. CV support, career guidance, workplace readiness, etc.), career programme level (can either be Foundation, Intermediate or Advanced), and lastly career programme capacity limit (the maximum number of students the programme can enrol).
- All the columns (attributes) cannot be null.
- car is short for career.
- A check constraint is used on carProgrammeCapacityLimit to make sure the value inserted is always positive.
- carAdvisorID links the career advisor table and career programme table together, since a career programme is facilitated (taught) by a career advisor, e.g. a career advisor who specialises in CV support will facilitate the CV writing programme.
- Refer to the foreign key specifications notes at the top of this section for how the foreign key constraint works. I also made sure a career advisor cannot be removed from the CAREER_ADVISOR table if they are still referenced here.
- This screen show shows that career_programme table has been created successfully



---

4. Creating PROGRAMME ENROLMENT Table

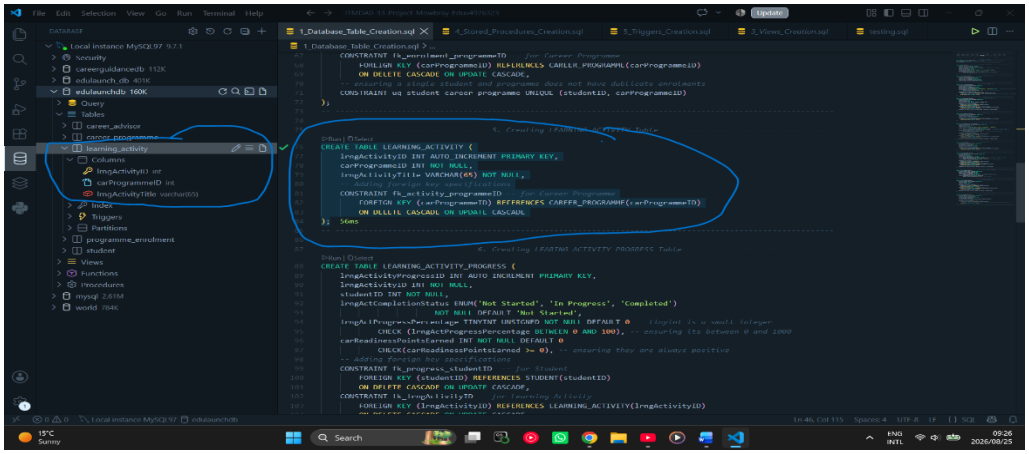
- This table stores programme enrolment details. Refer to section b, system specification 1, on the many-to-many relationship resolution part, to see why this table exists. This table resolves the many-to-many relationship between career programme and student.
- This code block creates the table (entity) PROGRAMME_ENROLMENT from the ERD with the following columns (attributes): programme enrolment ID (the primary key, auto generated), student ID (a foreign key from the student table), career programme ID (a foreign key from the career programme table), and lastly programme enrolment date (stores the date the enrolment was made, defaulting to the current date).
- studentID links the student table and programme enrolment table together, since a student enrolls in a career programme. carProgrammeID links the career programme table and programme enrolment table together, since an enrolment belongs to a career programme.
- All the columns (attributes) cannot be NULL.
- car is short for career, prgm is short for programme.
- Refer to the foreign key specifications notes at the top of this section for how the foreign key constraints work. I also added a constraint that makes sure studentID and carProgrammeID together are unique for each enrolment, meaning a student cannot enroll into the same career programme twice.
- This screen show shows that programme_enrolment table has been created successfully



---

5. Creating LEARNING ACTIVITY Table

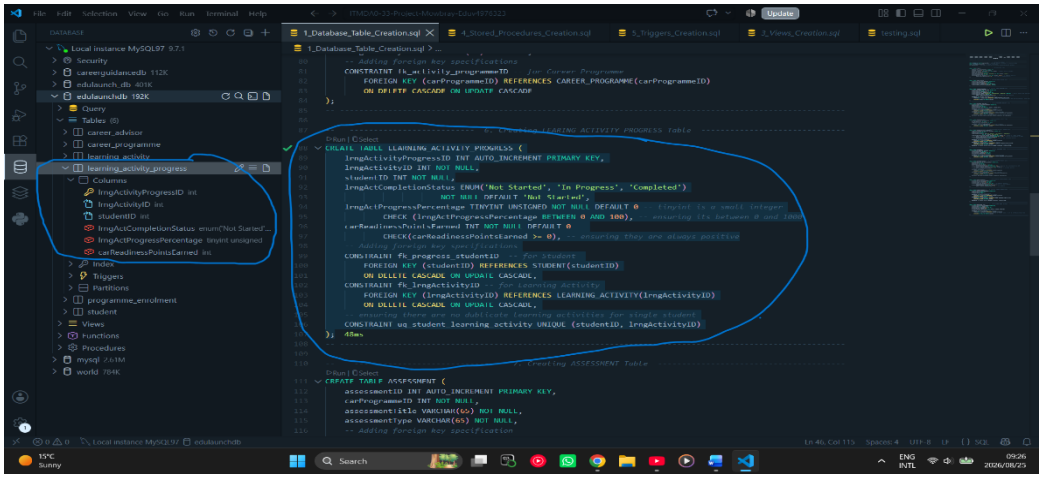
- This table will store learning activity details.
- This code block will create the table (entity) LEARNING_ACTIVITY from the ERD, with the following columns (attributes), learning activity ID (primary key that is auto generated), career programme ID (foreign key from the career programme table), and lastly learning activity title.
- Career programme ID links the career programme table and learning activity table together since each career programme can have zero or many learning activities.
- All the columns (attributes) here cannot be null.
- Lrng is short for learning.
- Refer to the notes on foreign key specification part at the top of this section to understand how the foreign key specification part works.
- This screen show shows that learning_activity table has been created successfully



---

6. Creating LEARNING ACTIVITY PROGRESS Table

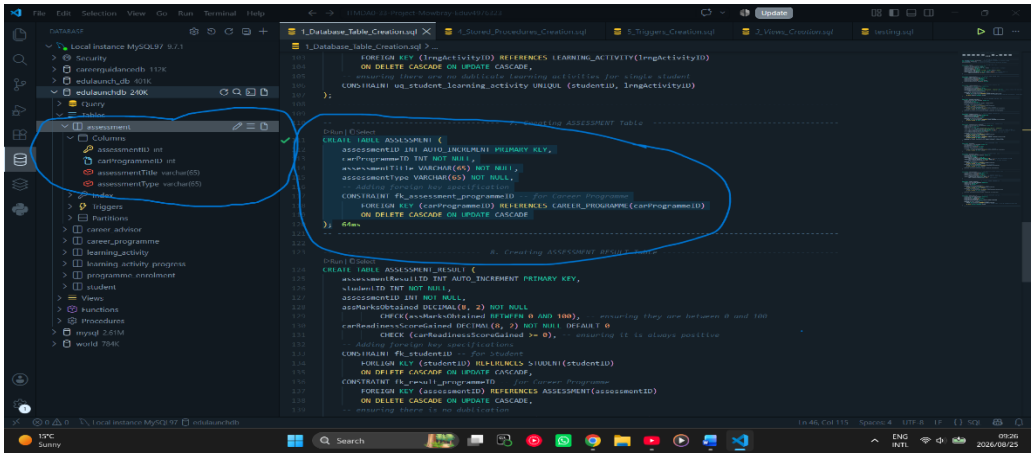
- This table will store learning activity progress details. Refer to section b, system specification 1 as to why this table exists on the many-to-many relationship resolution part. This essentially solves the many-to-many relationship between learning activity and student.
- This code block will create the table (entity) LEARNING_ACTIVITY_PROGRESS from the ERD, with the following columns (attributes), learning activity progress ID (primary key that is auto generated) learning activity ID (foreign key from the learning activity table), student ID (foreign key from the student table), learning activity completion status (can either be not started which is the default, in progress or completed), learning activity progress percentage, and lastly career readiness points earned.
- The check constraint is used to make sure that learning activity progress percentage is between 0 and 100 (default being 0), and to make sure that career readiness points are always positive (default being 0).
- All the columns (attributes) here cannot be null.
- Lrng is short for learning. Act is short for activity. Car is short for career.
- Refer to the notes on foreign key specification part at the top of this section to understand how the foreign key specification part works. I also added a constraint that makes sure that student ID and learning activity ID are unique for each activity progress, meaning a single activity progress can only belong to a single student and a single learning activity, removing duplication.
- This screen show shows that learning_activity_progress table has been created successfully



---

7. Creating ASSESSMENT Table

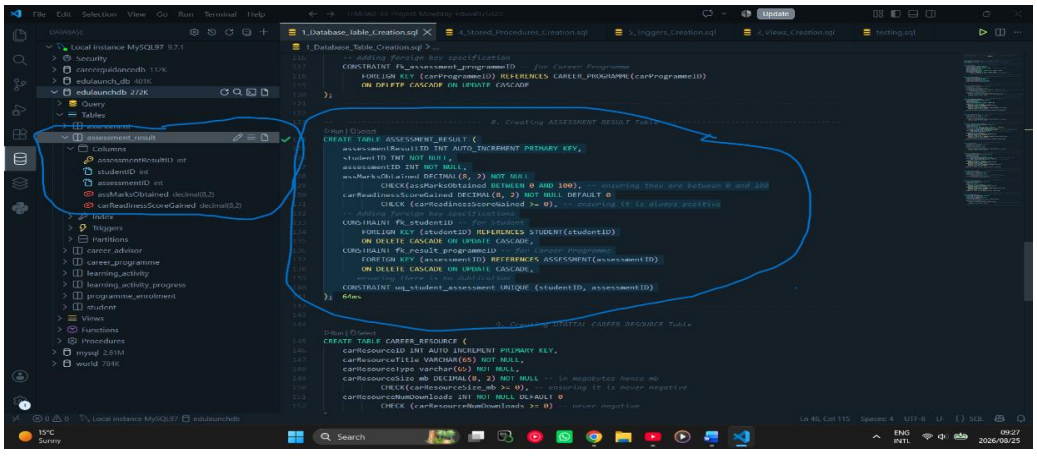
- This table with store assessment details.
- This code block will create the table (entity) ASSESSMENT from the ERD, with the following columns (attributes), assessment ID (primary key that is auto generated), career programme ID (foreign key from the career programme table), assessment title, assessment type (e.g. quiz, cv review, mock interview, etc.)
- Career programme ID links the career programme table and assessment table together since a career programme can have zero or many assessments.
- All the columns (attributes) here cannot be null.
- Car is short for career.
- Refer to the notes on foreign key specification part at the top of this section to understand how the foreign key specification part works.
- This screen show shows that assessment table has been created successfully
-



---

8. Creating ASSESSMENT RESULT Table

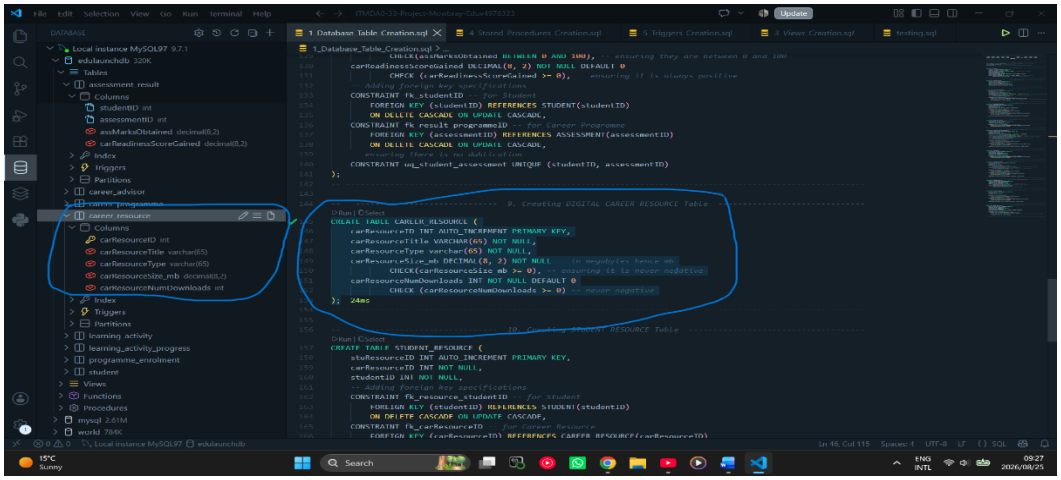
- This table will store assessment result details. Refer to section b, system specification 1 as to why this table exists on the many-to-many relationship resolution part. This essentially solves the many-to-many relationship between assessment and student.
- This code block will create the table (entity) ASSESSMENT_RESULT from the ERD, with the following columns (attributes), assessment result ID (primary key that is auto generated), student ID (foreign key from the student table), assessment ID (foreign key from the assessment table), assessment marks obtained, and lastly career readiness score gained.
- Student ID links the student table and assessment result table together since a student is the one that gets (or obtains) the result of the assessment.
- Assessment ID links the assessment table and assessment result together since an assessment result comes from an assessment.
- The check constraint is used to make sure that the marks obtained are between 0 and 100, and to also make sure that the career readiness score gained is positive (greater than or equal to 0, with the default being 0).
- All the columns (attributes) cannot be null.
- Car is short for career. Ass is short for assessment
- Refer to the notes on foreign key specification part at the top of this section to understand how the foreign key specification part works. I also added a constraint that makes sure student ID and assessment ID are unique for each assessment result, meaning each assessment result will belong to a single student and single assessment, removing duplication.
- This screen show shows that assessment_result table has been created successfully
-



---

9. Creating DIGITAL CAREER RESOURCE Table

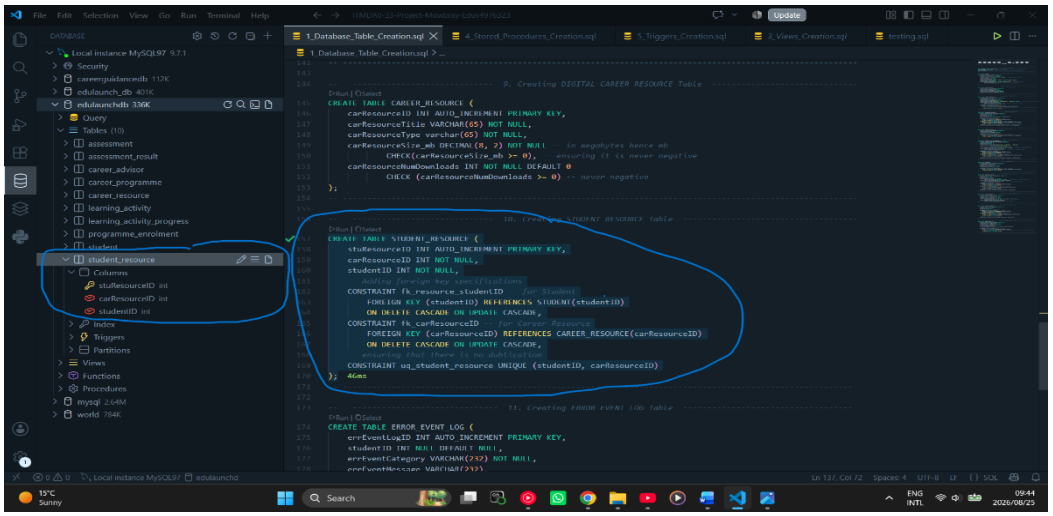
- This table will store career resource details.
- This code will create the table (entity) CAREER_RESOURCE from ERD, with the following columns (attributes), career resource ID (primary key that is auto generated), career resource title, career resource type (e.g. slides, video, pdf guide, etc.), career resource size (in megabytes), and lastly career resource number of downloads (default being 0).
- The check constraint is used to make sure that both career resource size and career resource number of downloads are always greater or equal to 0.
- All the columns (attributes) cannot be null.
- Car is short career. Num is short for number. Mb is short for megabytes.
- This screen show shows that career_resource table has been created successfully
-



---

10. Creating STUDENT RESOURCE Table

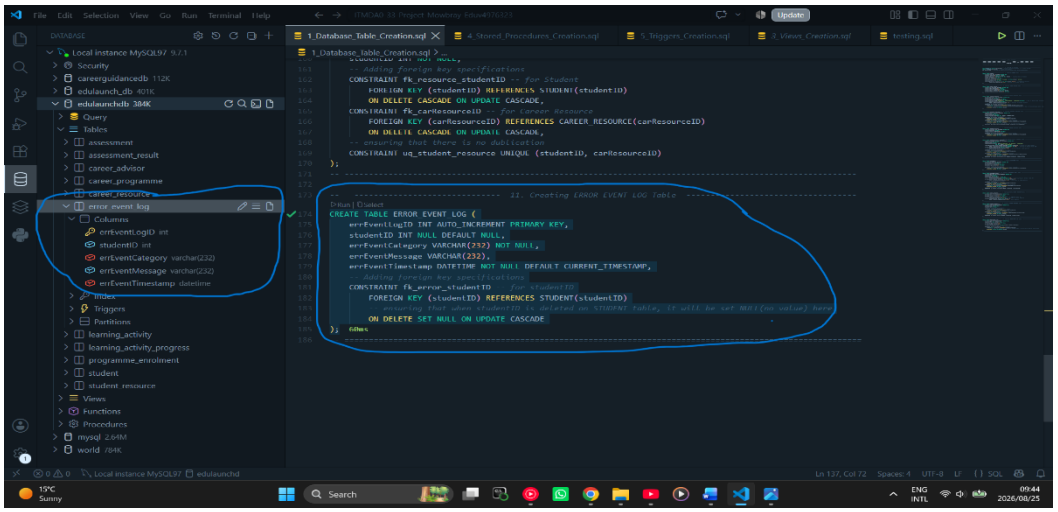
- This table will store students' downloaded resource details. Refer to section b, system specification 1 as to why this table exists on the many-to-many relationship resolution part. This essentially solves the many-to-many relationship between career resource and student.
- This code will create the table (entity) STUDENT_RESOURCE from ERD, with the following columns (attributes), student resource ID (primary key that is auto generated), career resource ID (foreign key from the career resource table), student ID (foreign key from the student table).
- Car is short for career. Stu is short for student.
- All the columns (attributes) cannot be null.
- Refer to the notes on foreign key specification part at the top of this section to understand how the foreign key specification part works. I also added a constraint that makes sure student ID and career resource ID are unique for each student's downloaded resource, meaning each student resource will belong to a single student and digital resource, meaning that a student will not download the digital resource again if they already have it stored.
- This screen show shows that student_resource table has been created successfully
-



---

11. Creating ERROR EVENT LOG Table

- This table will store error or events details.
- This code will create the table (entity) ERROR_EVENT_LOG from ERD, with the following columns (attributes), error event log ID (primary key that is auto generated), student ID, error event category (e.g. enrollment error, login failure, resource storage error, etc.), error even message, and lastly error event timestamp (default being the exact time the error happens)
- Err is short for error.
- Only student ID can be null because not all errors will be made by a student or registered student, the rest of the columns (attributes) cannot be null.
- I have student ID's default as null because if a student makes the error their student id has to be explicitly inserted otherwise there is no need to explicitly insert null.
- Refer to the notes on foreign key specification part at the top of this section to understand how the foreign key specification part works. When a student is removed/deleted on the student table, student ID here will be set null.
- This screen show shows that error_event_log table has been created successfully



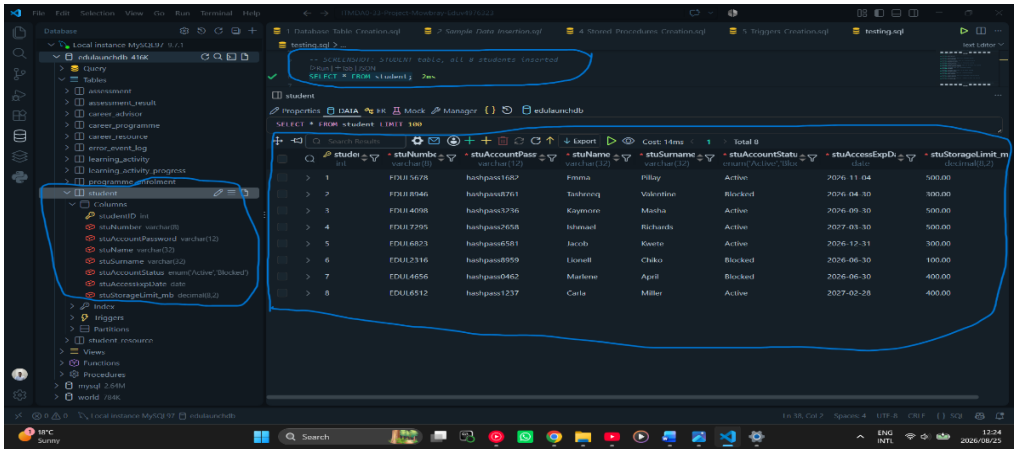
==========

2_Sample_Data_Insertion.sql

- **Notes**:
  - This should be run after 1_Database_Table_Creation.sql, it inserts data into the tables created inside edulaunchDB.
  - Although learning activities and assessments might look similar to each other (e.g. the learning activity Interview Practice and the assessment Mock Interview), they are not linked to each other anywhere in the database. Doing a learning activity might help a student in an assessment, but it does not mean that a student who skipped the learning activity will not get an assessment or fail it.

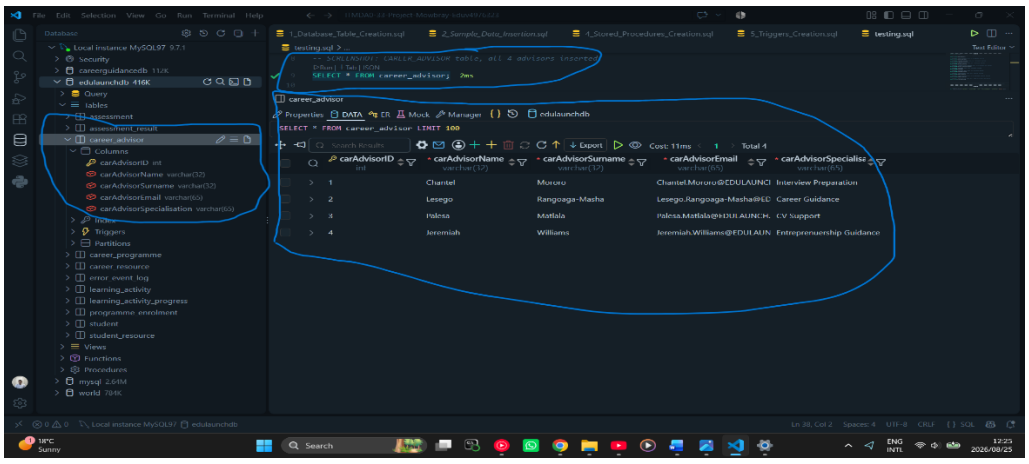
0. Using edulaunchDB
   - This makes sure these values are inserted into the tables inside the edulaunchDB database.

1. Inserting into STUDENT Table
   - This code block inserts 8 rows of data into the student table.
   - The values inserted are student number, student password, student name, student surname, student account status, student access expiry date, and lastly student storage limit.
   - The student number follows the EDULXXXX standard, with X being unique numbers, and the password follows the hashpassxxxx standard, with x being unique numbers, as explained in the previous script. This should be unique to every student.
   - I included varying student storage limits and access expiry dates for variability, and these rows represent both active and blocked students.
   - Jacob Kwete has a storage limit of 300 because I will later use this to test a procedure.
   - SCREENSHOT: STUDENT table, all 8 students inserted



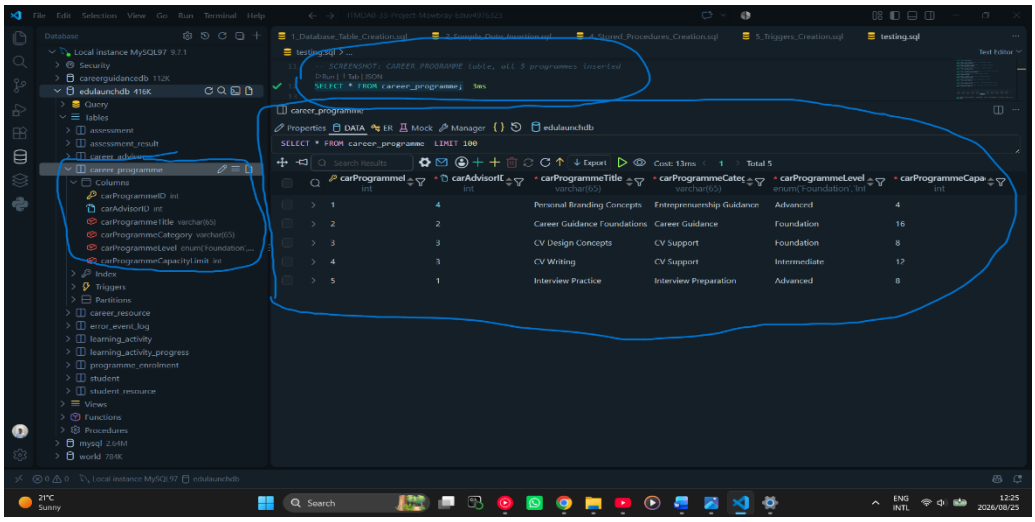
---

2. Inserting into CAREER ADVISOR Table
   - This code block inserts 4 career advisors into the career_advisor table.
   - The values inserted are career advisor name, career advisor surname, career advisor email address, and career advisor specialisation.
   - The career advisor email address follows the standard x.y@EDULAUNCH.COM, with x being their name and y being their surname.
   - SCREENSHOT: CAREER_ADVISOR table, all 4 advisors inserted



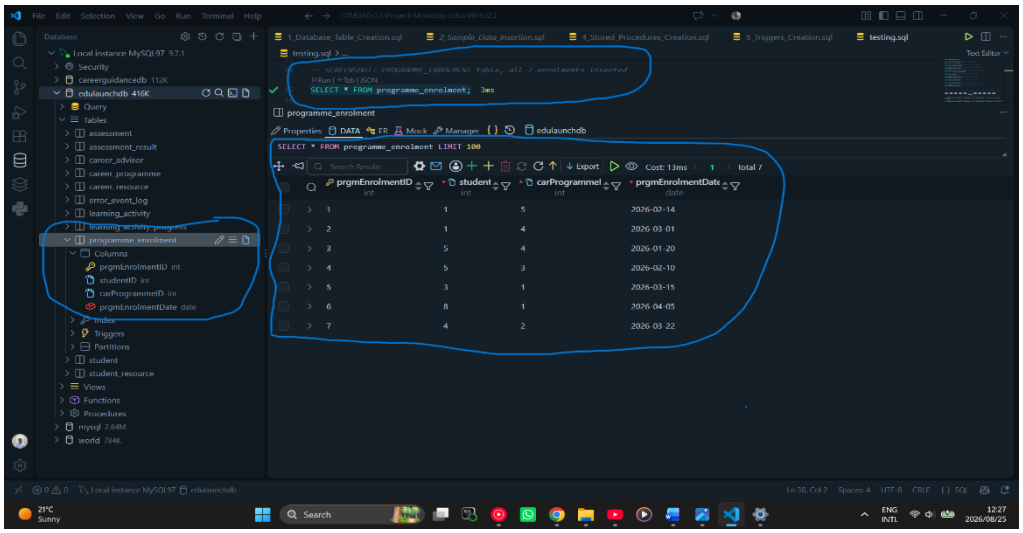
---

3. **Inserting into CAREER PROGRAMME Table**
   - This code block inserts 5 career programmes into the career_programme table.
   - The values inserted are career advisor ID (a foreign key from the career advisor table), career programme title, career programme category, career programme level, and career programme capacity limit.
   - Personal Branding Concepts has a capacity limit of 4 because I will later use it to test a procedure.
   - Notice how a single career programme category can have multiple career programmes under it.
   - SCREENSHOT: CAREER_PROGRAMME table, all 5 programmes inserted
   -



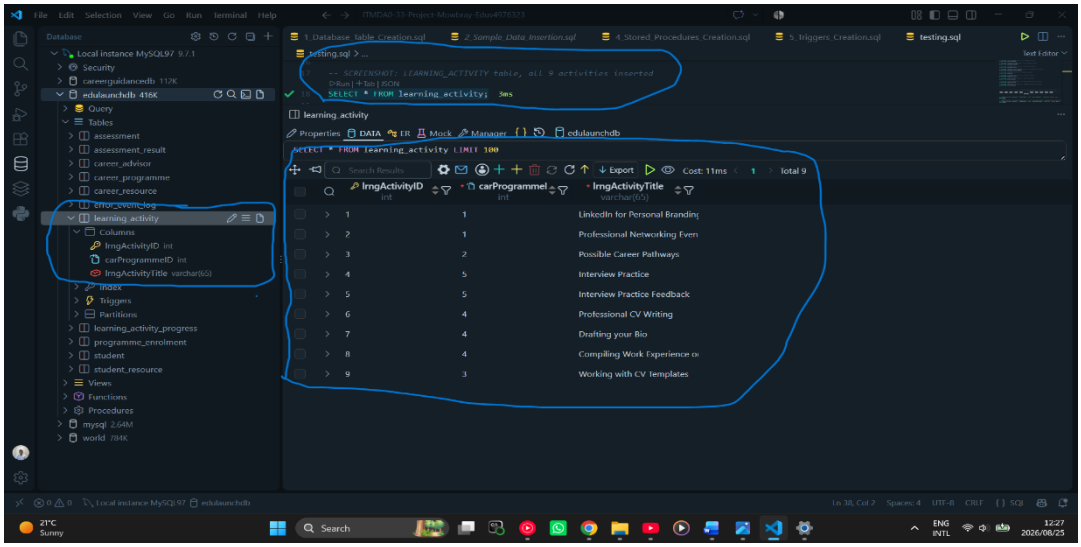
---

4. **Inserting into PROGRAMME ENROLMENT Table**
   - This code block inserts 7 rows into the programme_enrolment table.
   - The values inserted are student ID (a foreign key from the student table), career programme ID (a foreign key from the career programme table), and programme enrolment date.
   - Notice how a single student can be enrolled into multiple programmes.
   - Blocked students are not inserted here, because they cannot be enrolled into a programme if they no longer have access. Only active students appear in this table.
   - SCREENSHOT: PROGRAMME_ENROLMENT table, all 7 enrolments inserted
   -



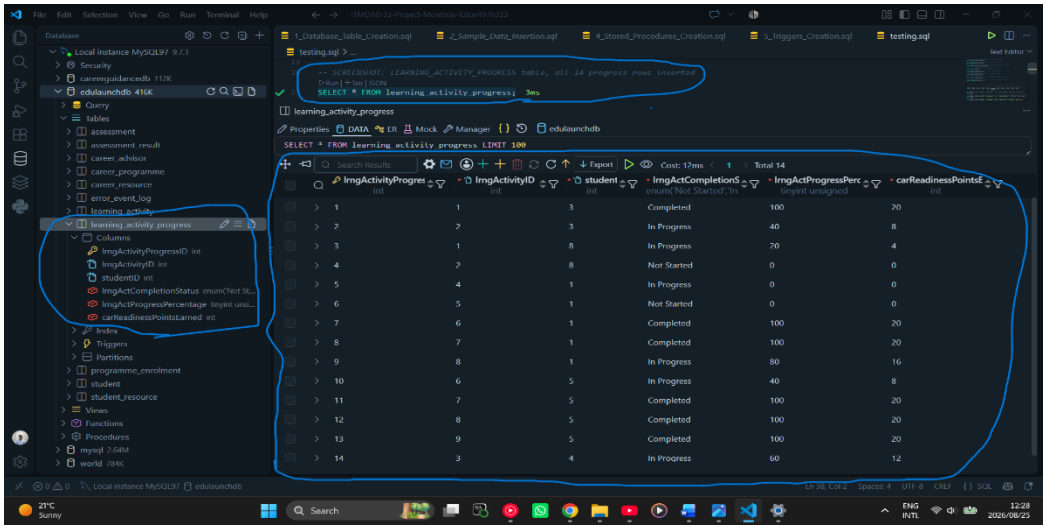
---

5. **Inserting into LEARNING ACTIVITY Table**
   - This code block inserts 9 rows into the learning_activity table.
   - The values inserted are career programme ID (a foreign key from the career programme table), and the learning activity title.
   - Each career programme has at least one learning activity attached to it.
   - A learning activity does not have to be a written activity, it can be a practical, a feedback session, a workshop, or even attending an event.
   - Compiling Work Experience or Education is an activity that helps students properly organise and write their work experience or education in a structured, professional way.
   - SCREENSHOT: LEARNING_ACTIVITY table, all 9 activities inserted
   -



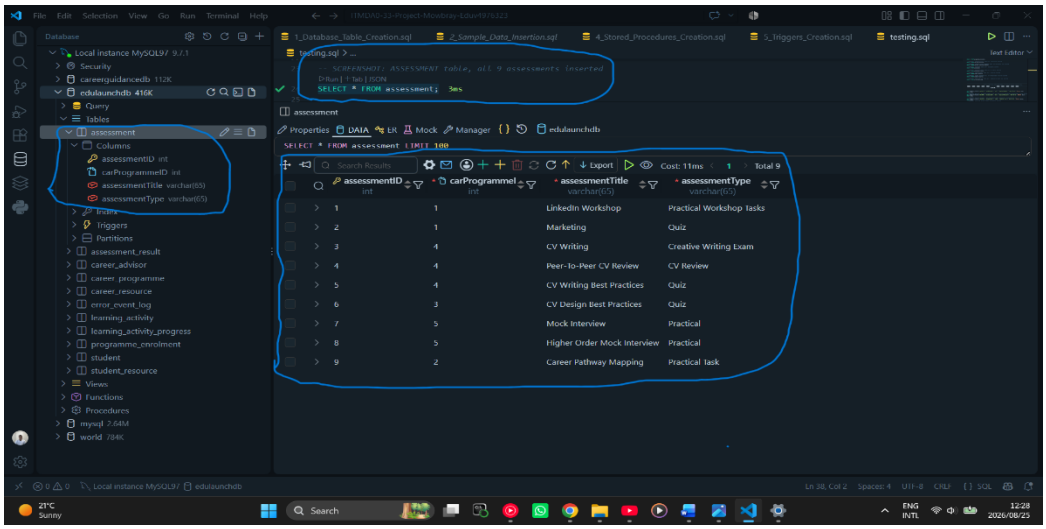
---

6. **Inserting into LEARNING ACTIVITY PROGRESS Table**.
   - This code block inserts 14 rows into the learning_activity_progress table.
   - The values inserted are student ID (a foreign key from the student table, since the student is the one making the progress), learning activity ID (a foreign key from the learning activity table, since the progress is for a learning activity), learning activity completion status, learning activity progress percentage, and lastly career readiness points earned from that activity.
   - If a student has not started the learning activity, the points earned are 0. The progress percentage moves in 5s (5, 10, 15 ...) to avoid decimal points, meaning for every 5% of progress a student makes, they earn 1 career readiness point. The math is [learning activity progress percentage / 5 = career readiness point(s) earned]. I know this makes things a little more complicated, but I like it that way.
   - SCREENSHOT: LEARNING_ACTIVITY_PROGRESS table, all 14 progress rows inserted
   -



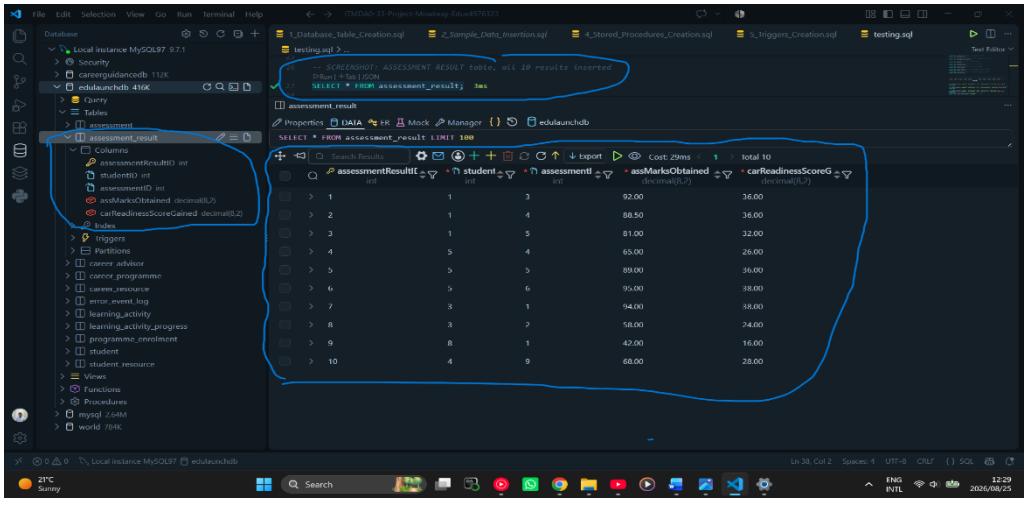
---

7. Inserting into ASSESSMENT Table

- This code block inserts 9 rows into the assessment table.
- The values inserted are career programme ID (a foreign key from the career programme table, since each career programme has assessments), assessment title, and lastly assessment type.
- A career programme can have multiple assessments, and assessment type varies from quizzes, to reviews, to workshops, to practicals, etc.
- SCREENSHOT: ASSESSMENT table, all 9 assessments inserted
- 

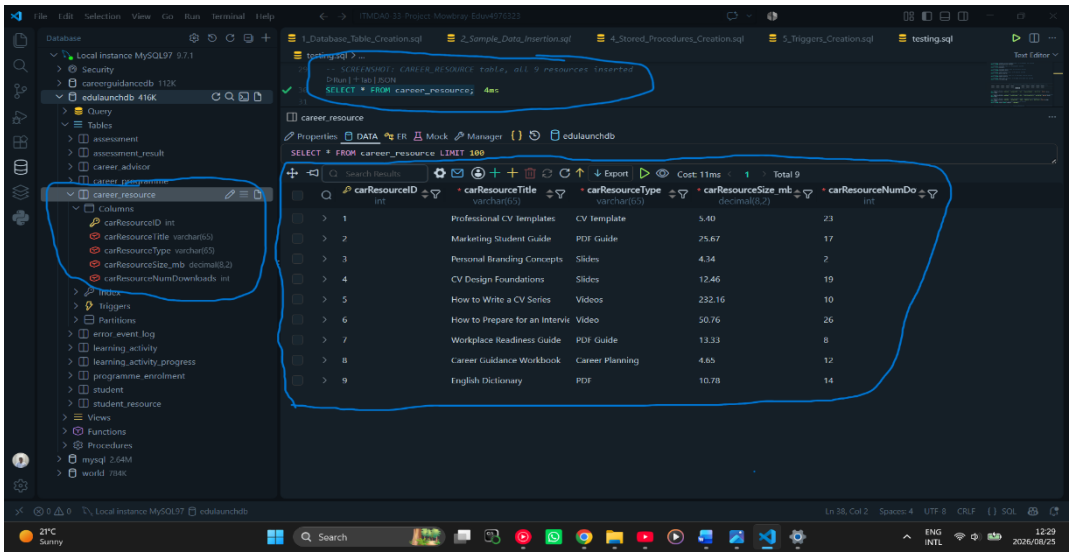
---

8. Inserting into ASSESSMENT RESULT Table

- This code block inserts 10 rows into the assessment_result table.
- The values inserted are student ID (a foreign key from the student table, since the student is the one who gets the result), assessment ID (a foreign key from the assessment table, since these are results for that assessment), assessment marks obtained, and lastly career readiness score gained.
- The career readiness score gained is calculated in 5s as well, similar to the learning activity progress percentage, but this time every 5 marks percentage a student obtains earns them 2 points. Since marks obtained run from 0 to 100 moving in 1s and not in 5s, each mark a student obtains is rounded to the nearest 5 only to work out the score gained, not to change the actual mark recorded. The math is [marks obtained rounded to the nearest 5 / 5 * 2 = career readiness score gained], e.g. a student who obtains 67 marks rounds to 65, so the readiness score gained is 65 / 5 * 2, which is 26.
- SCREENSHOT: ASSESSMENT_RESULT table, all 10 results inserted
- 

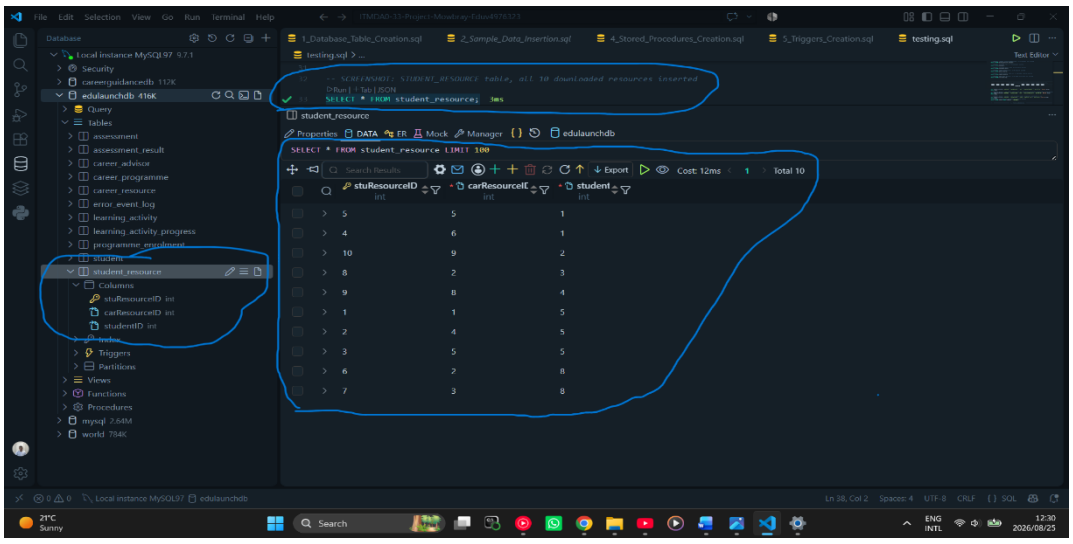
---

9. Inserting into CAREER RESOURCE Table

- This code block inserts 9 rows into the career_resource table.
- The values inserted are career resource title, career resource type, career resource size in megabytes, and career resource number of downloads.
- A student's ability to download a career resource is limited by how much storage they have available, and they cannot download the same resource twice if they already have it downloaded, they would have to delete the existing one first.
- How to Write a CV Series is a series of videos, which is why its size is 232.16 MB, I did this on purpose because I will use it later to test that a student cannot download a career resource once their storage limit has been reached.
- SCREENSHOT: CAREER_RESOURCE table, all 9 resources inserted
- 

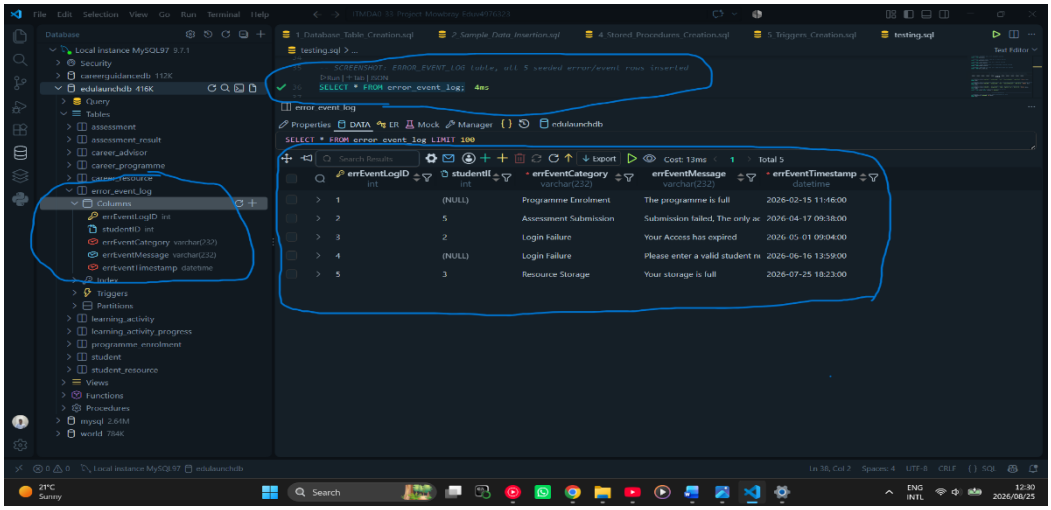
---

10. Inserting into STUDENT RESOURCE

- This code block inserts 10 rows into the student_resource table.
- The values inserted are student ID (a foreign key from the student table, since the student is the one downloading the resource into their personal storage), and career resource ID.
- There is not much more to this code block than adding digital career resources into a student's personal storage, the same storage rules explained for the career_resource table apply here too.
- SCREENSHOT: STUDENT_RESOURCE table, all 10 downloaded resources inserted
- 

---

11. Inserting into ERROR EVENT LOG Table

- This code block inserts 5 rows into the error_event_log table.
- The values inserted are student ID (a foreign key from the student table, and this can be NULL), error event category, error event message, and lastly error event timestamp.
- The first row has a NULL student ID on purpose, to show that errors are not only ever logged by an existing, registered student. Should a student cause the error though, their student ID should be stored.
- SCREENSHOT: ERROR_EVENT_LOG table, all 5 seeded error/event rows inserted
- 

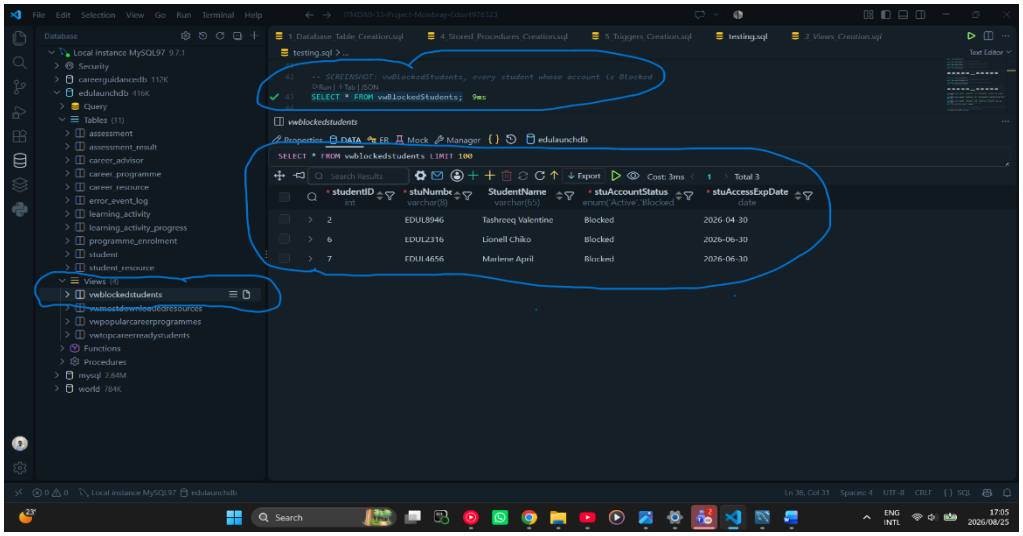
==========

3_Views_Creation.sql

- **Notes:**
  - This should be run after 2_Sample_Data_Insertion.sql.
  - Views are virtual, table-like structures that give a representation of data, they do not store the data themselves but fetch it every time they are queried. They act like a SELECT statement that is predefined, so instead of always writing out the same SELECT statement every time we want to check, say, blocked students, we can create a view that stores that SELECT statement and just query the view instead.
  - AS is used to create an alias, which is a temporary name assigned to a column (attribute) or a table. The alias used for every table in these views is its initials, e.g. s for student, cr for career resource, etc.
  - CREATE OR REPLACE means and does exactly what those words say, it is used to create a view, and if a view with that name already exists, it replaces it.
  - CONCAT adds things together, SELECT selects a column (attribute), and FROM specifies which table (entity) to select it from. LIMIT limits the results to the number specified, e.g. LIMIT 20 limits the results to 20 rows, LIMIT 5 limits the results to 5 rows.
  - GROUP BY groups the selected rows in the result set into summarised groups, and is normally used together with aggregate functions like SUM, COUNT and AVG.
  - A LEFT JOIN uses the left table (the one mentioned in the FROM statement) as the reference, meaning every row on the left table will show in the result, and rows from the right table are matched to it. If the right table has no match for a row on the left table, NULL is returned for that value instead.
  - COALESCE returns the first value in a list that is not NULL.

1. Creating View vwBlockedStudents

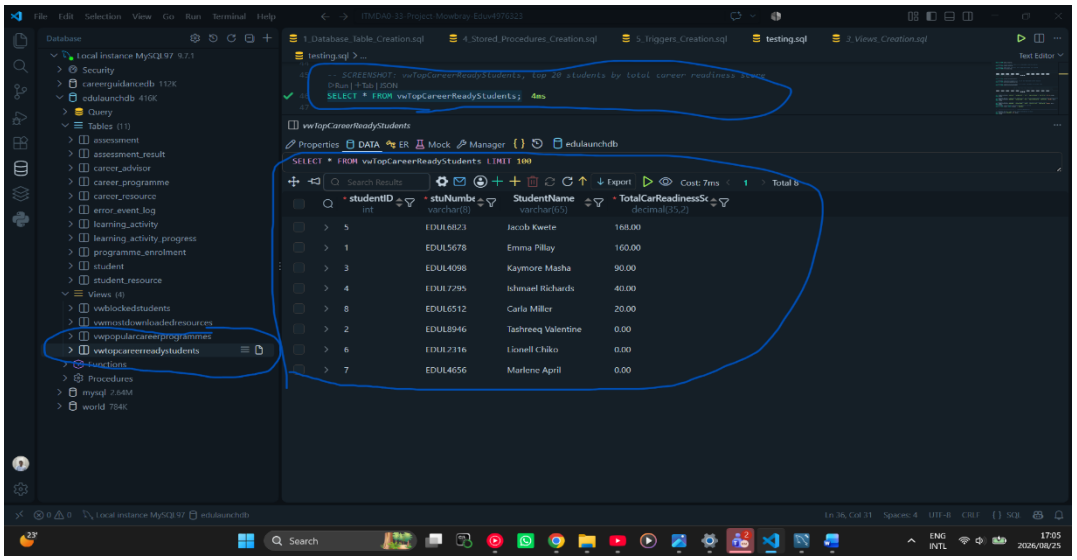
- This view stores a predefined SELECT statement that returns every student whose account status is Blocked.
- This code block returns each blocked student's student ID, student number, name and surname (combined together and aliased as StudentName), account status, and lastly their access expiry date.
- CONCAT adds the student's name and surname together, aliased as StudentName.
- Both conditions in the WHERE clause need to be true for a student to be returned: the student's account status must be Blocked, and the student's access expiry date must be less than today's date. If either condition is not true, that student is left out of vwBlockedStudents.
- This view makes it easy to search for and find blocked students, which makes it easier to, for example, renew their access.
- The results are ordered by ascending order of access expiry date.
- This screenshot shows the result of the view vwBlockedStudents.
-



---

2. Creating View vwTopCareerReadyStudents

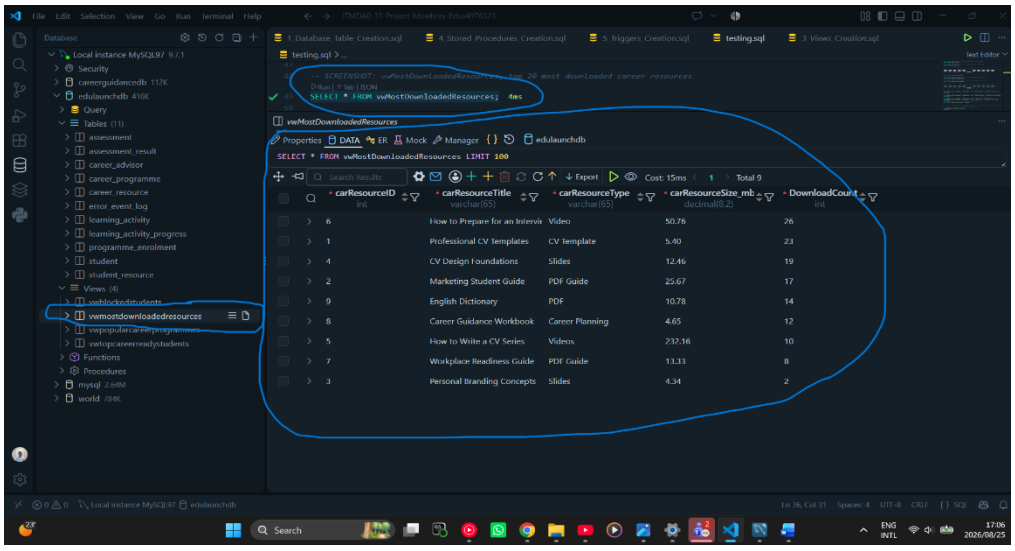
- This view stores a predefined SELECT statement that returns the top 20 students with the highest career readiness score, based on both the career readiness points and the career readiness score a student has gained.
- This code block returns each of the top 20 students' student ID, student number, name and surname (aliased as StudentName), and the total of their career readiness score from assessment results (aliased as TotalAssessmentScore) and their career readiness points from learning activities (aliased as TotalActivityPoints) added together as TotalCarReadinessScore.
- Both tables are left joined onto the student table on studentID, and I chose LEFT JOIN because it keeps every row on the student table and matches in whatever it can find from the assessment result and learning activity progress tables.
- COALESCE turns any NULL summed-up career readiness score or points into 0, refer to the coalesce note above for how coalesce works. SUM adds up all the points and scores a student has earned across every learning activity and assessment result they have.
- I chose to add career readiness points gained from learning activities together with career readiness score from assessment results, instead of using only one of them, because otherwise students would not be incentivised to do the learning activities, and those that do them should be rewarded for it.
- The results are the top 20 students, ordered by descending order of their total career readiness score.
- This screenshot shows the result of the view vwTopCareerReadyStudents:



---

3. Creating View vwMostDownloadedResources

- This view stores a predefined SELECT statement that returns the top 20 most downloaded career resources.
- This code block returns each of the top 20 most downloaded career resources' career resource ID, title, type, size in megabytes, and its number of downloads (aliased as DownloadCount).
- The results are the top 20, ordered by descending order of DownloadCount.
- This screenshot shows the result of the view vwMostDownloadedResources:

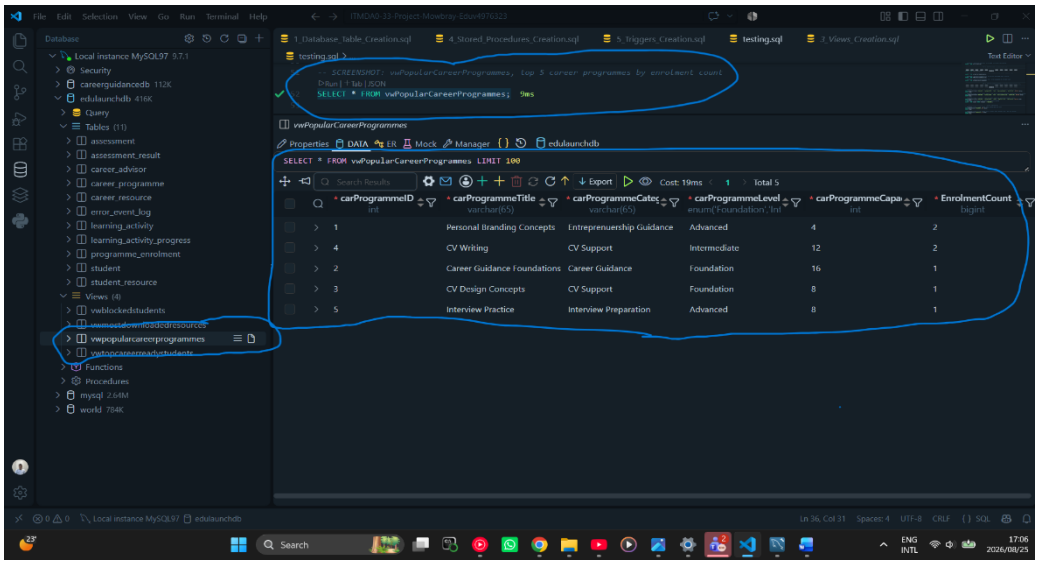


-

---

4. Creating View vwPopularCareerProgrammes

- This view stores a predefined SELECT statement that returns the top 5 career programmes based on enrolment count.
- This code block returns each of the top 5 career programmes' ID, title, category, level, capacity limit, and the count of all their programme enrolment rows (aliased as EnrolmentCount).
- LEFT JOIN is used to keep every row from the career programme table (the left table) and match it to the programme enrolment table (the right table), since the number of enrolments can only be found on the programme enrolment table.
- The results are the top 5 career programmes, ordered by descending order of EnrolmentCount.
- This screenshot shows the result of the view vwPopularCareerProgrammes
-



==========

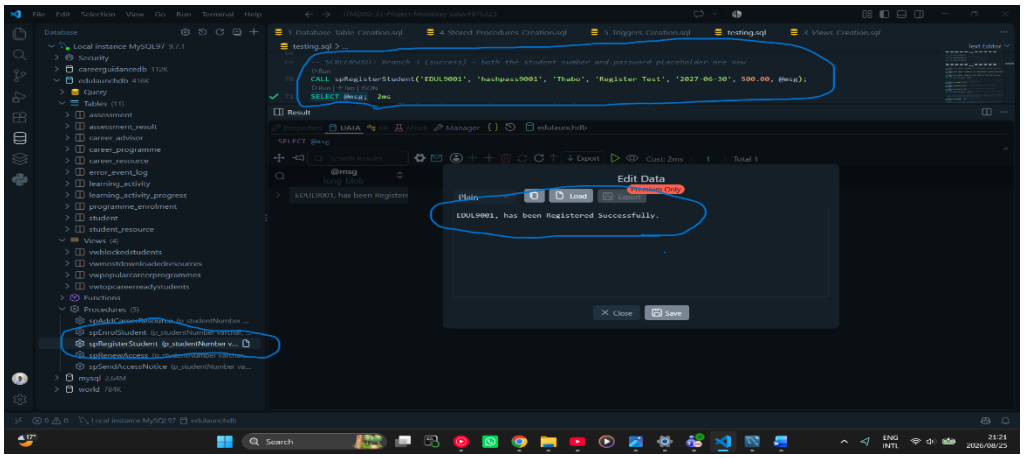
4_Stored_Procedures_Creation.sql

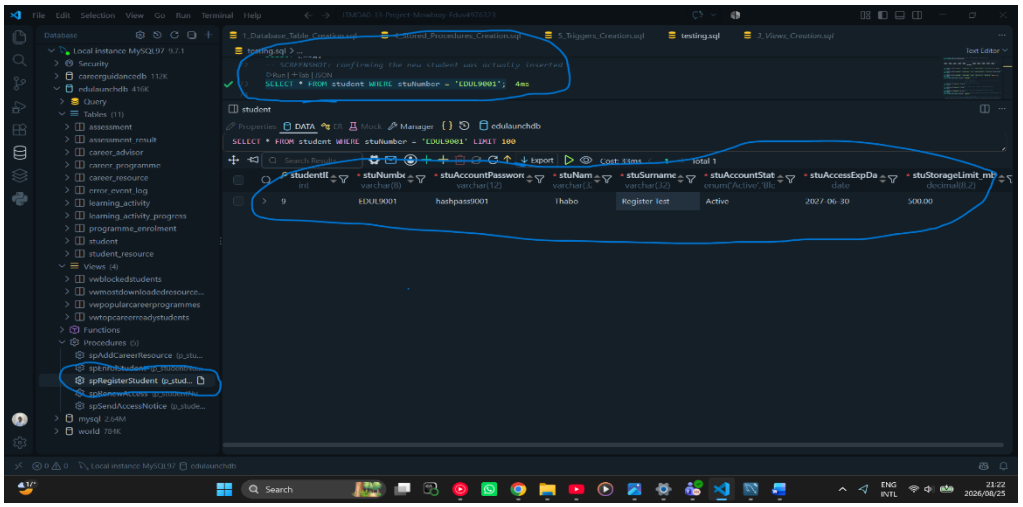
- **Notes**
  - This should be run after 3_Views_Creation.sql.
  - A Stored Procedure is the storing of a combination of SQL statements that are used frequently, for easy accessibility and time saving. So, if you find yourself writing the same SQL statements over and over again, it is a good idea to turn those SQL statements into a procedure that gets you the result you want without having to write the SQL statements again. Think of it like when you always use onions when you cook, instead of cutting them every single time, you cut a lot of them at once, store them somewhere, and whenever you need them, you just take from where you stored them instead of always cutting them again.
  - Procedures are used with the SQL key word CALL, e.g. CALL spEnrolStudent().
  - In order to see the OUT message then use CALL the procedure, then input the required parameters of that procedure and also add @msg as one of the parameters, then select the @msg.

    e.g. CALL spRenewAccess('EDUL0000', 30 @message); SELECT @msg;
  - DELIMITER is what tells SQL that we are done with a block of code, the default one being a semi-colon (;). In a stored procedure, we change the delimiter to something else (normally // or $$), because a stored procedure contains multiple SQL statements inside it, so having a semi-colon at the end of every SQL statement inside the procedure would confuse SQL, because it would then think we are done with the whole block of code every single time. Changing the delimiter to something like // tells SQL that we will have multiple statements ending in a semi-colon inside, and that this does not mean we are done with the entire group of statements yet. What tells SQL we are done with the entire procedure is the // delimiter.
  - IN is an input that the user gives SQL. OUT is an output that SQL gives back to the user on the console.
  - BEGIN tells SQL that we are starting something. DECLARE declares a variable (a storage of something), which will store what we want it to store, and can be given a DEFAULT starting value.
  - IF is a conditional statement that tells SQL that if x happens, then do y. THEN tells SQL what to do if the condition is true. ELSEIF is used when there are multiple conditions, so if the first IF is false, SQL moves to the ELSEIF, and if that is true, that happens, if false, SQL moves to the next condition. You can tell SQL what to do if every IF and ELSEIF is false with an ELSE condition, which cannot stand on its own, there must be an IF condition before it. You can only have one IF and one ELSE, but there is no limit to how many ELSEIFs you can have.
  - CASE WHEN works similarly to IF/ELSEIF/ELSE, except it is used inside of a SELECT statement instead of inside a procedure's logic, to return a different value depending on the condition. WHEN checks a condition, THEN says what to return if that condition is true, ELSE says what to return if none of the WHEN conditions were true, and END closes off the CASE statement.
  - SET sets a value to a variable that has been declared. INTO puts something from somewhere into whatever it is meant to go into.
  - COUNT(*) counts how many rows a query returns. This gets used a lot in these procedures as a way of checking whether something exists, e.g. if COUNT(*) returns more than 0, that means at least one matching row was found.
  - DATEDIFF works out the difference, in days, between two dates that are given to it.
  - I chose to use the student's number instead of the student's ID where possible, because every student has a unique student number, meaning it also qualifies as something that identifies an instance (the student), and the chances of an administrator knowing a student's student number instead of their student ID is very high. So a student number makes more sense here, and it saves time since the administrator would not have to look up the auto incremented student ID first.
  - The p in p_x (e.g. p_studentNumber) stands for parameter, which is a place holder that passes a value into or out of a procedure. The v in v_x (e.g. v_studentID) stands for variable, which is a storage of something.
  - DATE_ADD is an SQL function that adds dates together, GREATEST chooses the largest option out of two things given to it, CURRENT_DATE is today's date, and INTERVAL specifies a span of time (day, week, month or year).
  - The screenshot of all the error logs logged by the stored procedures is at the end.

0. Using EdulaunchDB and Setting Delimiter

- This makes sure the stored procedures are created inside edulaunchDB.
- The delimiter is also changed to //. Refer to the delimiter note above to understand why this is important.

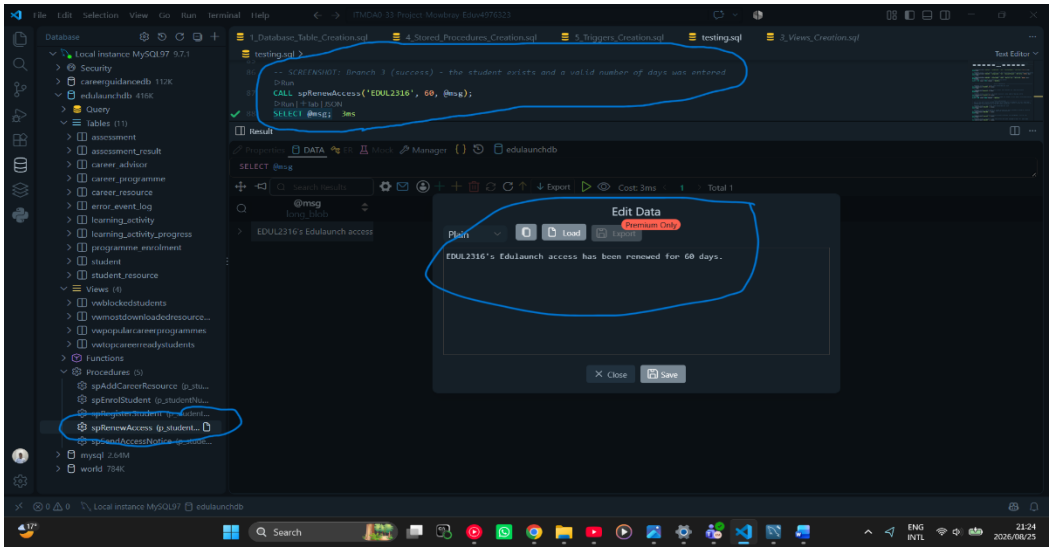
1. **Creating Stored Procedure spRegisterStudent**
   - This procedure registers a new student into the system, and makes sure the student number and the password placeholder entered are not already being used by another student before adding them.
   - This code block needs the user to input (in) the student number (p_studentNumber), student password (p_studentPassword), student name (p_studentName), student surname (p_studentSurname), student access expiry date (p_studentAcessExpDate), and student storage limit (p_studentStorageLimit_mb), and will output (out) a message (p_message) to the console depending on what happens.
   - I declare two variables, v_existingStuNumber and v_existingStuPassword, both defaulting to 0, which store whether the student number or password placeholder entered already exists on the student table.
   - I select the count of all (*) students where the student number matches the input p_studentNumber into v_existingStuNumber, and the count of all students where the student password matches the input p_studentPassword into v_existingStuPassword.
   - If v_existingStuNumber is greater than 0, it means a student with that student number already exists, then log the error into the error event log table and set the out message to explain that the student number already exists in the system.
   - Elseif v_existingStuPassword is greater than 0, it means the password placeholder entered is already being used by another student, then log the error into the error event log table and set the out message to explain that the password placeholder already exists in the system. I checked for this because stuAccountPassword must be unique on the student table, so registering a student with a password placeholder that is already used would fail, and I want to catch that and explain why before it happens.
   - Else, meaning the student number and password placeholder are both free to use, then register (insert) the new student into the student table using all the input values, and set the out message to say the student has been registered successfully.
   - End if and end the procedure with //.
   - This screenshot shows the message of success that a learner has been registered successfully:
   - 

   - This screenshot shows that the student is inserted into student table:
   - 

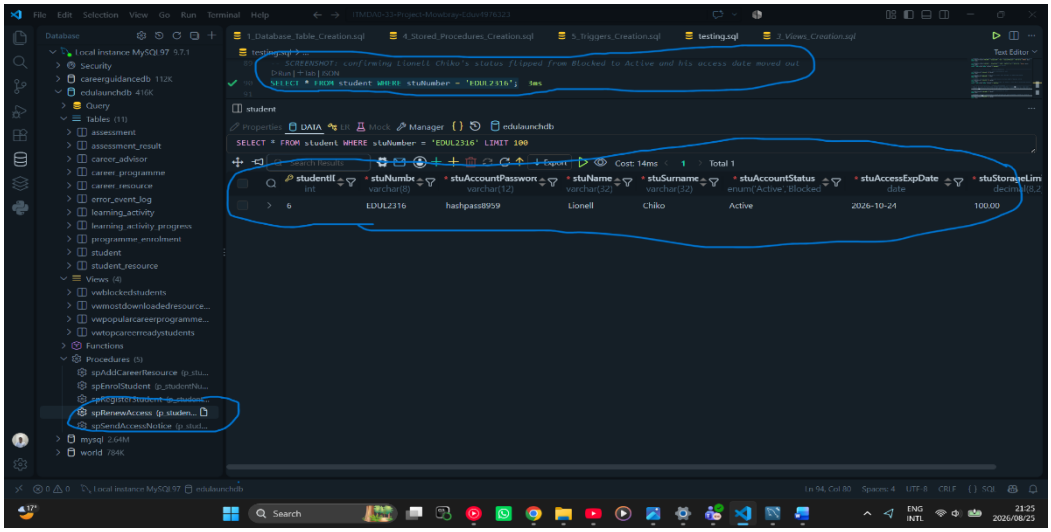
---

2. Creating Stored Procedure spRenewAccess

- This procedure adds days to a student's EduLaunch access expiry date, renews access for blocked students, and changes the student's account status from Blocked to Active once their access has been renewed.
- This code block needs the user to input (in) the student number (p_studentNumber) and the number of days to add (p_numDaysToAdd) to that student's access date and will output (out) a message (p_message) to the console depending on what happens.
- I declare a variable called v_studentID, which stores the student's ID from the student table, then I select studentID into that variable, where the student number matches the input p_studentNumber.
- If the student ID is not found (is null), the out message is set to say the student number does not exist. Elseif the number of days to add is 0 or less, the then out message is set to say the number of days needs to be higher than 0. Else, update the student table by setting the student's access expiry date to the number of days to add, starting from today's date, and change the student's account status to Active, then set the out message to say access has been renewed. End the if statements and the whole procedure with //.
- I set the new student access expiry date using the date_add function, adding the latest date, chosen with greatest (between when the access already expired and today's date), with an interval of the input number of days to add, specified as days.
- This screenshot shows the message of success that a student's access has been renewed successfully:



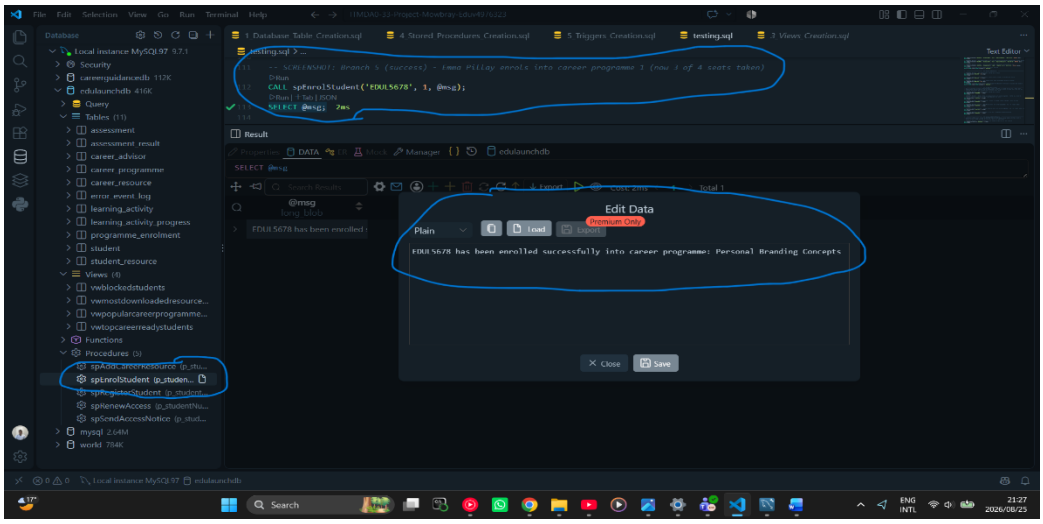
- This screenshot shows that a student's access has been renewed and status is active from the student table:



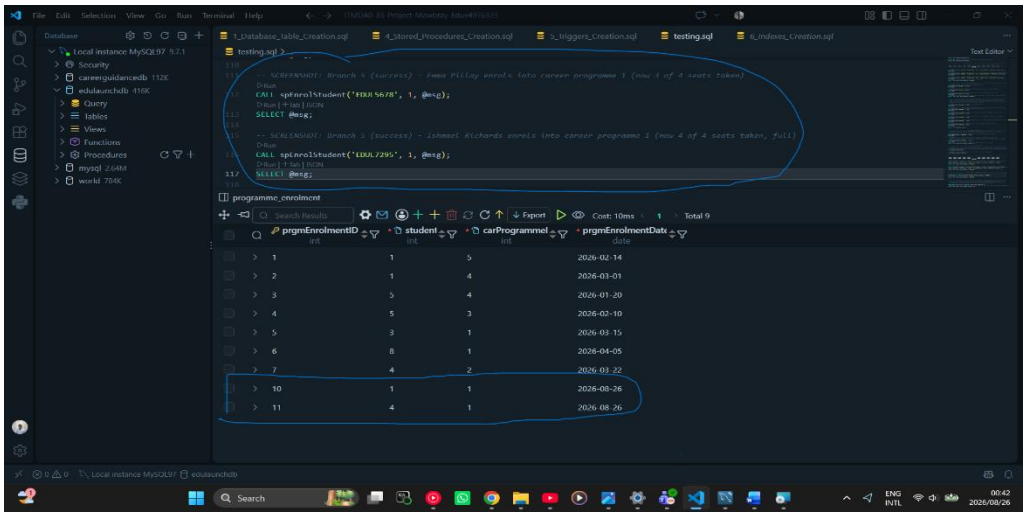
---

3. Creating Stored Procedure spEnrolStudent

- This procedure enrolls a student into a career programme.
- This code block needs the user to input (in) the student number (p_studentNumber) of the student to enroll, and the ID of the career programme (p_careerProgrammeID) to enroll them into and will output (out) a message on the console depending on what happens.
- I declare the variables v_prgmCapacityLimit (career programme capacity limit), v_carProgrammeTitle (career programme title), v_currentlyEnrolled (how many students are currently enrolled), v_accountStatus (the student's account status), v_alreadyEnrolled (whether the student is already enrolled into this career programme), and lastly v_studentID (the student's ID).
- I select the career programme capacity limit and title into v_prgmCapacityLimit and v_carProgrammeTitle from the career programme table, where the career programme ID matches p_careerProgrammeID; the count of all (*) students currently enrolled into v_currentlyEnrolled from the programme enrolment table, where the career programme ID matches p_careerProgrammeID; the student's account status and ID into v_accountStatus and v_studentID from the student table, where the student number matches p_studentNumber; and lastly the count of all the times this student is already enrolled into this career programme into v_alreadyEnrolled, from the programme enrolment table, where both the student ID matches v_studentID and the career programme ID matches p_careerProgrammeID.
- If v_studentID or v_prgmCapacityLimit comes back null, it means the student number or the career programme entered do not exist, then the out message says one or both of these do not exist.
- Elseif v_currentlyEnrolled is greater than or equal to v_prgmCapacityLimit, it means the career programme is full, then log the error into the error event log table and set the out message to say the career programme is full.
- Elseif v_alreadyEnrolled is greater than 0, it means the student is already enrolled into this career programme, then I log the error into the error event log table and set the out message to say the student is already enrolled.
- Elseif v_accountStatus is not Active, I log the error into the error event log table and set the out message to say the student's account status is not active.
- Else, meaning the career programme and student both exist, the programme is not full, the student is not already enrolled, and the student's account is active, insert (enroll) the student into the programme enrolment table using the student's ID (v_studentID) and the inputted career programme ID (p_careerProgrammeID), and set the out message to say the student has been enrolled successfully into the career programme, showing the career programme's title (v_carProgrammeTitle).
- End if and end the procedure with //.
- This screenshot shows the success message of enrolling a student:

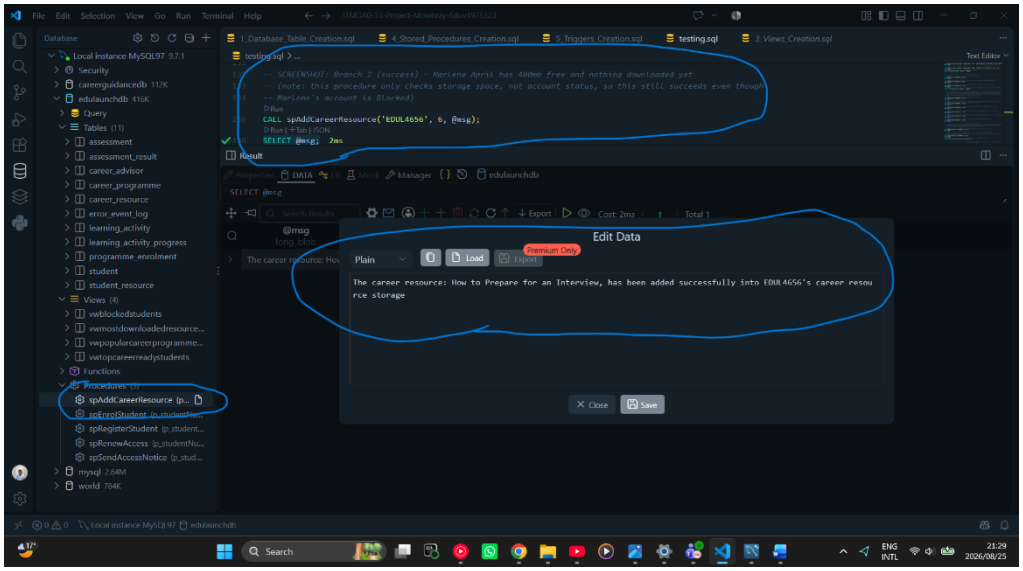


- This screenshot shows that the student has been enrolled successfully from the enrollment tabl:

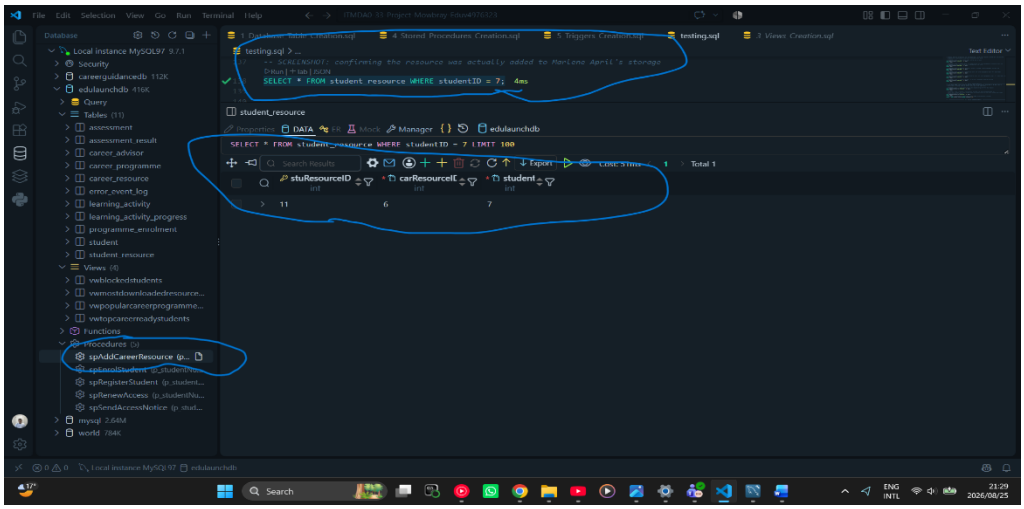


4. Creating Stored Procedure spAddCareerResource

- This procedure adds (downloads) a digital career resource into a student's personal storage and makes sure the student has enough storage space left before adding it.
- This code block needs the user to input (in) the student number (p_studentNumber) and the career resource ID (p_careerResourceID) of the resource being added and will output (out) a message (p_message) to the console depending on what happens.
- I declare the variables v_stuStorageLimit_mb (the student's total storage limit), v_stuUsedStorage_mb (how much storage the student has already used), v_studentID (the student's ID), v_newCarResourceSize_mb (the size of the resource being added), and v_carResourceTitle (the title of the resource being added).
- I select the student's storage limit and student ID into v_stuStorageLimit_mb and v_studentID from the student table, where the student number matches p_studentNumber.
- I select the coalesce of the sum of every career resource size the student already has downloaded, then join student_resource and career_resource on carResourceID, where studentID matches v_studentID, into v_stuUsedStorage_mb. If coalesce turns a null result into 0, for a student that has not downloaded anything yet, refer to the coalesce note in the views creation section to see how coalesce works.
- I select the size and title of the career resource being added into v_newCarResourceSize_mb and v_carResourceTitle, from the career resource table, where the career resource ID matches p_careerResourceID.
- If the new resource's size added to the student's already used storage is more than the student's storage limit, it means the student does not have enough storage space left, then log the error into the error event log table and set the out message to say the resource could not be added because the student does not have enough storage.
- Else, meaning the student has enough storage space left, insert (download) the resource into the student resource table using the student's ID and the input career resource ID, and set the out message to say the resource has been added successfully into the student's storage.
- End if and end the procedure with //.
- This screenshot shows the success message of adding a career resource into a student's personal storage:



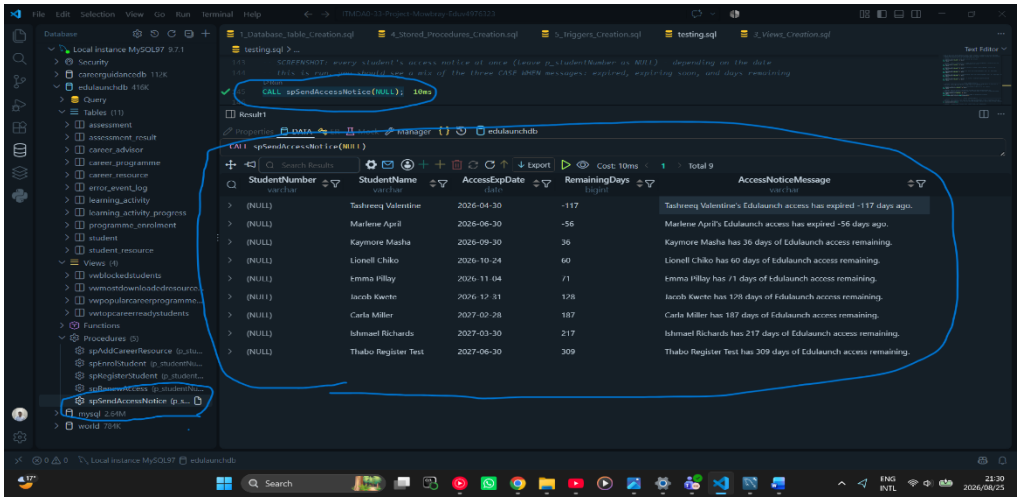
- This screenshot shows that the resource has been inserted successfully in the student resource table.



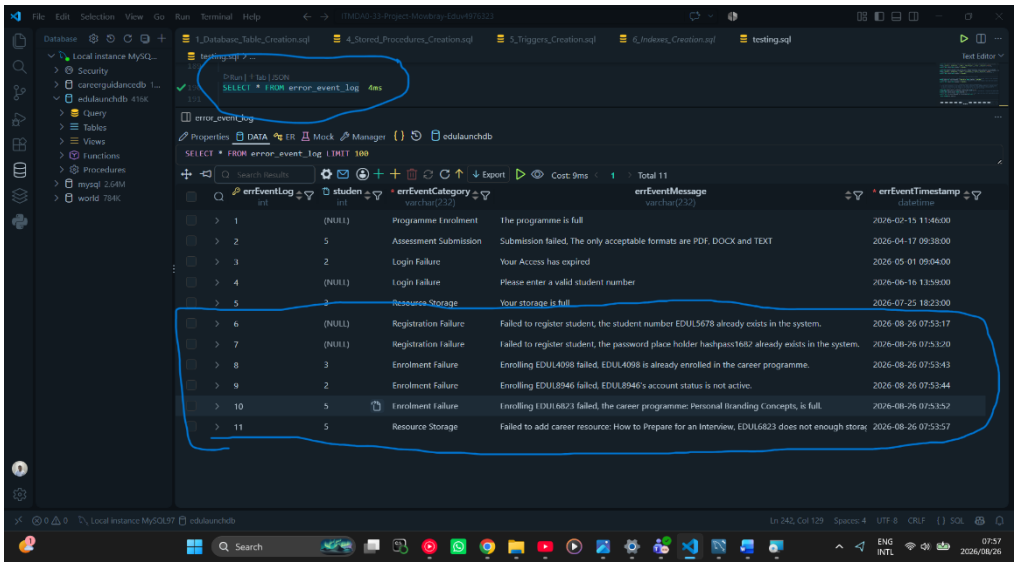
---

5. Creating Stored Procedure spSendAccessNotice

- This procedure shows how many days of EduLaunch access students have left and returns a notice message depending on how close (or how expired) their access is. It can either be run for a single student, or for every student if no student number is given.
- This code block needs the user to input (in) the student number (p_studentNumber), but this input can be left as null, in which case the procedure runs for every student instead of just one.
- Unlike the other procedures, this one does not use an out message parameter. It directly returns a result set (a table of rows) using select, since this procedure is meant to be read as a report rather than to give back a single pass or fail outcome.
- This code block selects: the student number (aliased as StudentNumber), the student's name and surname added together (aliased as StudentName), the student's access expiry date (aliased as AccessExpDate), the number of days remaining until access expiry using datediff (aliased as RemainingDays), and lastly a notice message built with a case when statement (aliased as AccessNoticeMessage).
- Datediff works out the difference, in days, between the student's access expiry date and today's date (current_date()), which tells us how many days are left before access expires. If the date has already passed, datediff returns a negative number.
- The first when checks if the student's account status is Blocked, and if it is true, then the message says the student's access has already expired, showing how many days ago.
- The second when checks if the remaining days are 30 or less, and if it is true, then the message says the student's access is expiring soon, showing how many days are left.
- If neither of the above are true, meaning the student is Active and has more than 30 days left, the else message says how many days of access the student has remaining.
- The where clause checks if the input student number (p_studentNumber) is null, or if it matches a student's stuNumber, meaning if nothing is entered when calling the procedure, every student is returned, but if a student number is entered, only that student is returned.
- The results are ordered by ascending order (asc) of RemainingDays, meaning students who are closest to running out of access, or who are already blocked, show first.
- The delimiter is reset back to a semi-colon with DELIMITER ; at the end of the script.
- This screenshot shows that spSendAccessNotice has been successfully sent to every student:

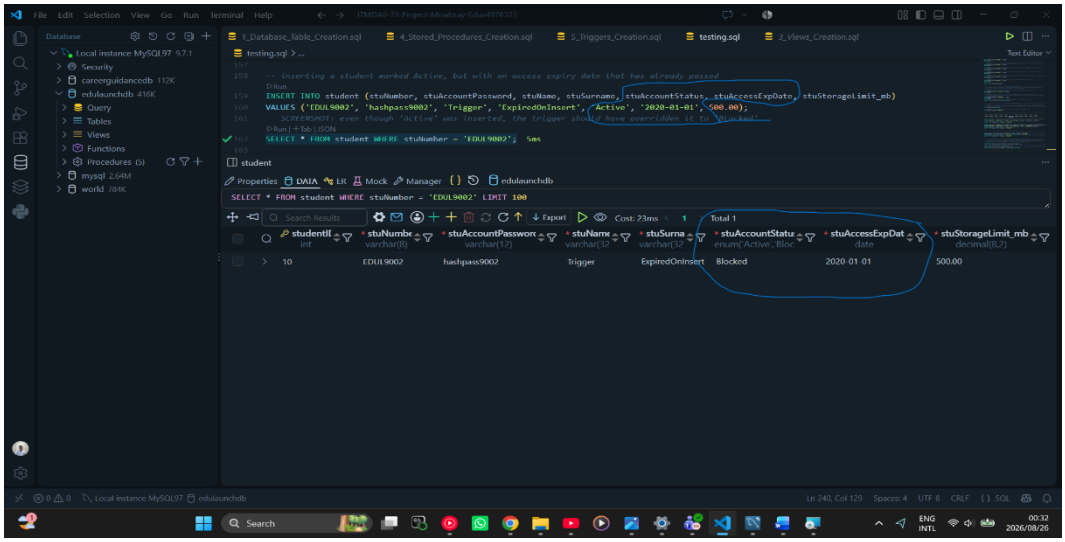


- This screenshot shows all the errors logged by all the stored procedures using the IF statements.



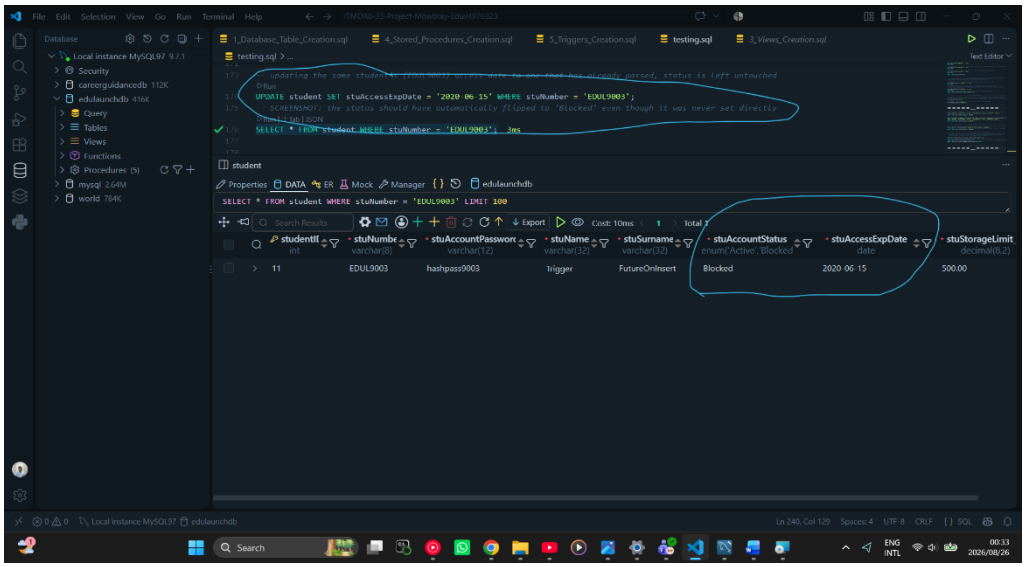
5_Trigger_Creating.sql

- **Notes**
  - This should be run after 4_Stored_Procedures_Creation.sql.
  - A Trigger is a block of SQL code that automatically runs by itself whenever a specific action (INSERT, UPDATE or DELETE) happens on a table, without anyone having to call it, unlike a stored procedure.
  - BEFORE means the trigger runs before the action happens (e.g. before a row is actually inserted or updated), which means the trigger can still change the values before they get saved. AFTER means the trigger runs after the action has already happened (e.g. after a row has already been inserted), which means the trigger can no longer change what was just saved, but it can still react to it, e.g. by logging something.
  - FOR EACH ROW means the trigger runs once for every single row affected by the action, instead of only once for the whole action.
  - NEW refers to the new values of the row being inserted or updated. NEW is only used on INSERT and UPDATE triggers, since there is no “new” row on a DELETE.
  - DELIMITER works the same way here as it does in stored procedures, refer to the delimiter note in the stored procedures creation section.

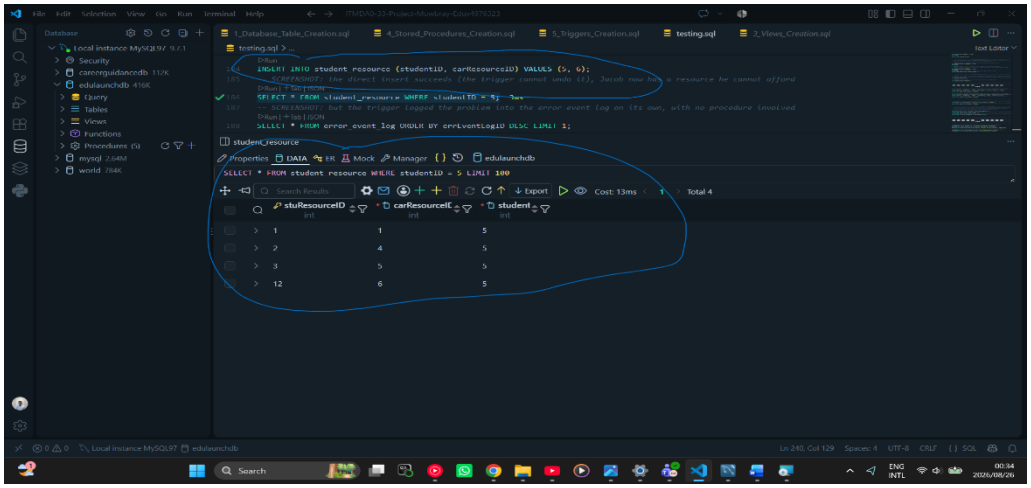
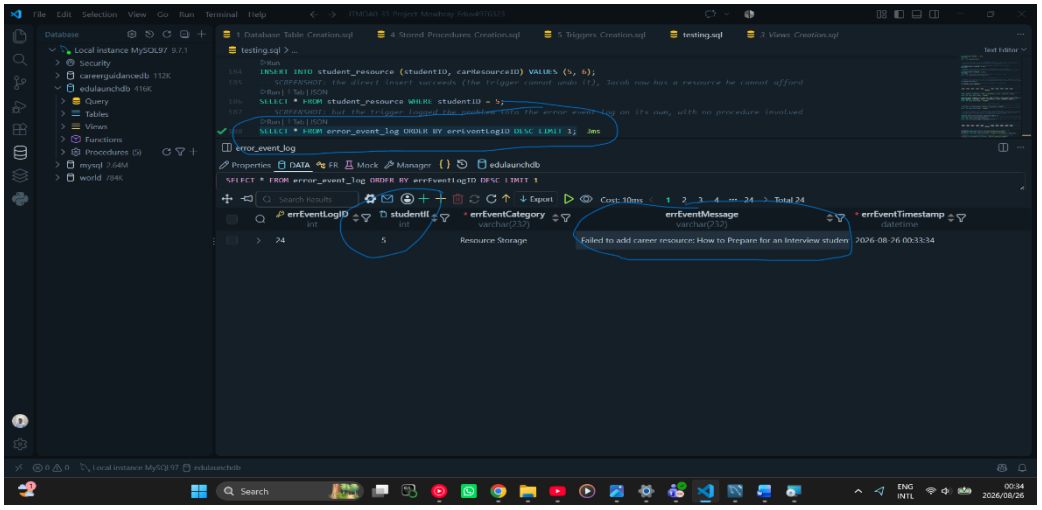
1. **Creating Trigger trg_BFL_AutoBlock_Student**
   - This trigger automatically blocks a student's account before they are even inserted into the student table, if their access has already expired.
   - This runs BEFORE INSERT on the student table, FOR EACH ROW, meaning it checks every new student being added.
   - If the new row's (NEW) access expiry date is before today's date (CURRENT_DATE()), the new row's account status is set to Blocked before it gets inserted, overriding whatever account status value was given.
   - This makes sure a student can never accidentally be registered as Active with an access expiry date that has already passed.
   - This screenshot shows me inserting a student using with a date that has already passed and they are automatically blocked even though I inserted their status as active:
   - 

---

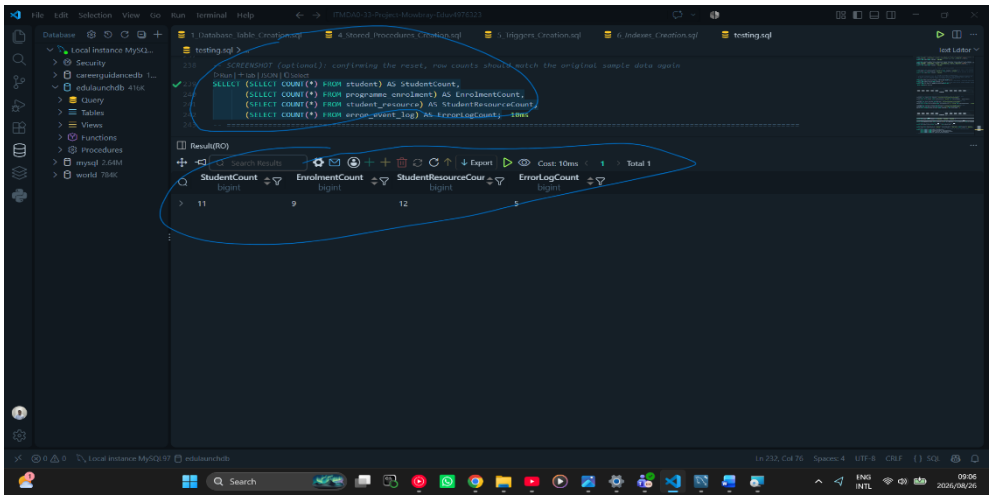
2. **Creating Trigger trg_BFU_AutoBlock_Student**
   - This trigger works exactly like trg_BFI_AutoBlock_Student, except it runs on UPDATE instead of INSERT.
   - This runs BEFORE UPDATE on the student table, FOR EACH ROW.
   - If the updated row's (NEW) access expiry date is before today's date, the row's account status is set to Blocked before the update is saved.
   - This makes sure that if a student's access expiry date is ever updated to a date in the past, or a student's record is edited in some other way, their account is automatically blocked, instead of relying on someone remembering to change the status by hand.
   - This screenshot shows me updating a student with a date that has already passed and they are automatically blocked even though I their status was originally active:
   -



---

3. **Creating Trigger trg_AFI_LogError_StudentResource**
   - This trigger automatically logs an error into the error event log table if a career resource is added directly into the student_resource table, bypassing spAddCareerResource, and the student did not actually have enough storage space for it.
   - This runs AFTER INSERT on the student_resource table, FOR EACH ROW, meaning it checks every new resource being added to a student's storage, after it has already been added.
   - I declared the same four variables used in spAddCareerResource: v_stuStorageLimit_mb, v_stuUsedStorage_mb, v_newCarResourceSize_mb, and v_carResourceTitle.
   - I select the student's storage limit into v_stuStorageLimit_mb, from the student table, where studentID matches the new row's (NEW) studentID.
   - I select the COALESCE of the SUM of every career resource size the student has downloaded into v_stuUsedStorage_mb, joining student_resource and career_resource, where studentID matches the new row's studentID. This total already includes the resource that was just inserted, since the trigger runs after the insert.
   - I select the size and title of the resource that was just added into v_newCarResourceSize_mb and v_carResourceTitle, from the career resource table, where carResourceID matches the new row's carResourceID.
   - If the new resource's size added to the student's used storage is more than the student's storage limit, log the error into the error event log table, showing the resource's title and that the student does not have enough storage.
   - I added this trigger because spAddCareerResource already checks this same storage limit before adding a resource, but that check can only stop someone if they actually go through the procedure. This trigger acts as a safety net that still catches and logs the problem if a resource is ever inserted into student_resource directly, without going through spAddCareerResource.
   - Note that unlike spAddCareerResource, this trigger only logs the problem, it does not undo (roll back) the insert.
   - This screenshot shows me inserting a career resource into a student's library who does not have enough storage, so this shows that the career resource has not been added:
   - 
   - This screenshot shows the error being logged in the error log table:
   - 

- This screenshot shows that I deleted all the changes I made to test the procedure and reverted back to the original values that I inserted at the beginning:



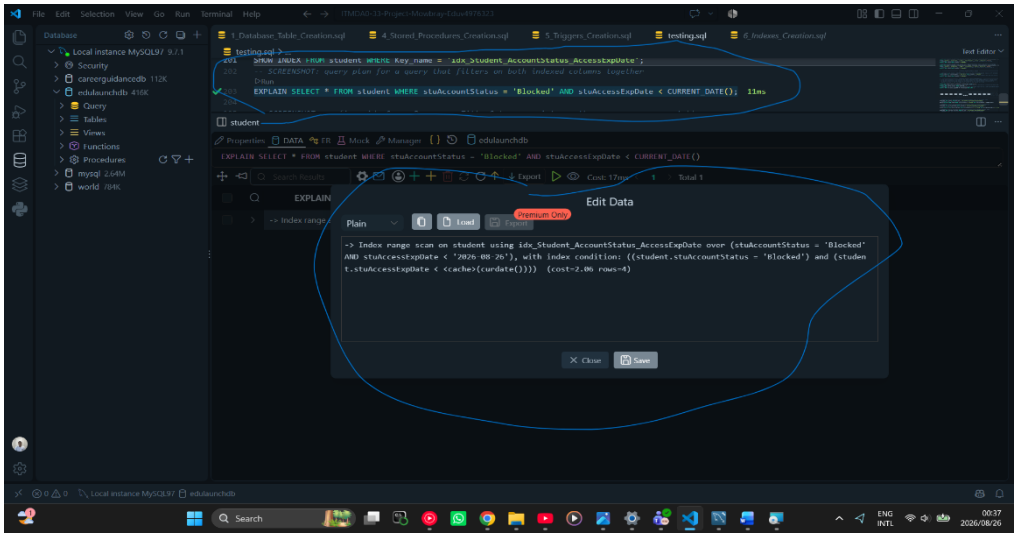
-

==========

6_Indexes_Creation.sql

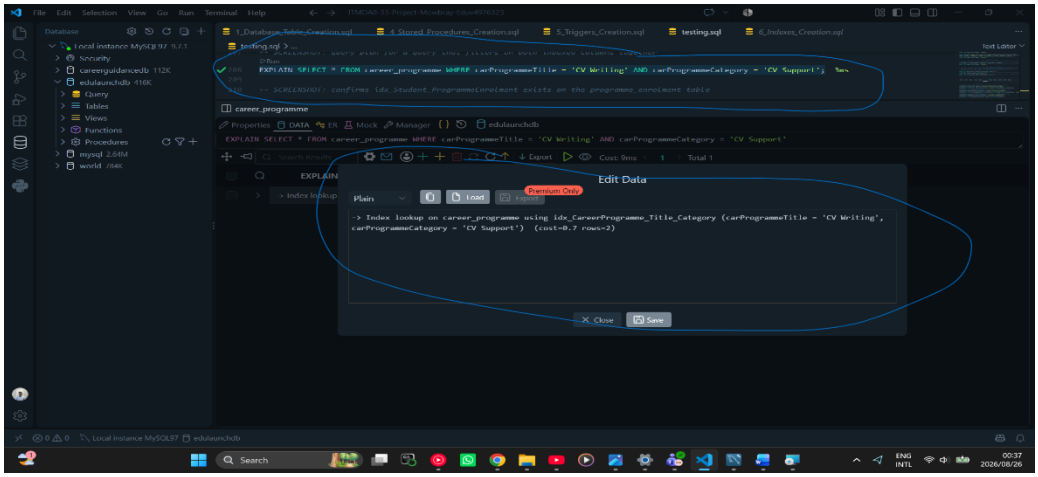
- **Notes**
  - This should be run after 5_Triggers_Creation.sql.
  - An Index is like the index at the back of a book. Instead of MySQL having to look through every single row in a table to find what it is looking for (this is called a full table scan), it can use the index to jump straight to the rows that match, which makes searching, filtering and sorting a lot faster, especially as a table grows bigger.
  - CREATE INDEX idx_x ON table (column1, column2, ...) creates an index named idx_x, on the columns listed, for the table given.
  - Indexes work best on columns that are frequently used in WHERE clauses, JOIN conditions, or ORDER BY clauses, since those are the operations an index actually speeds up.
  - idx is short for index.

1. **Creating Index idx_Student_AccountStatus_AccessExpDate**
   - This index is created on the student table, on the stuAccountStatus and stuAccessExpDate columns together.
   - This speeds up any query that filters students by their account status and access expiry date together, e.g. the vwBlockedStudents view, which filters for students that are Blocked and whose access expiry date is before today, and spSendAccessNotice, which reads every student's access expiry date and account status to build its notice message.
   - This screenshot shows how the index idx_Student_AccountStatus_AccessExpDate works using explain:
   -



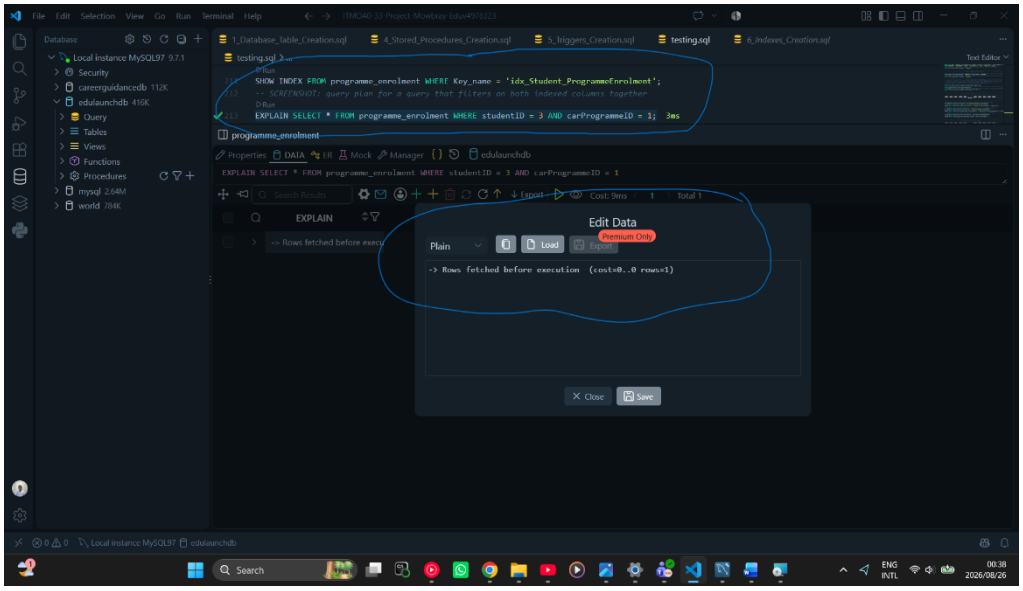
---

2. **Creating Index idx_CareerProgramme_Title_Category**
   - This index is created on the career_programme table, on the carProgrammeTitle and carProgrammeCategory columns together.
   - This speeds up searching for or filtering career programmes by their title or category, e.g. if an administrator wants to find every career programme under the CV Support category, or search for a programme by its title.
   - This screenshot shows how the index idx_CareerProgramme_Title_Category works using explain:
   -



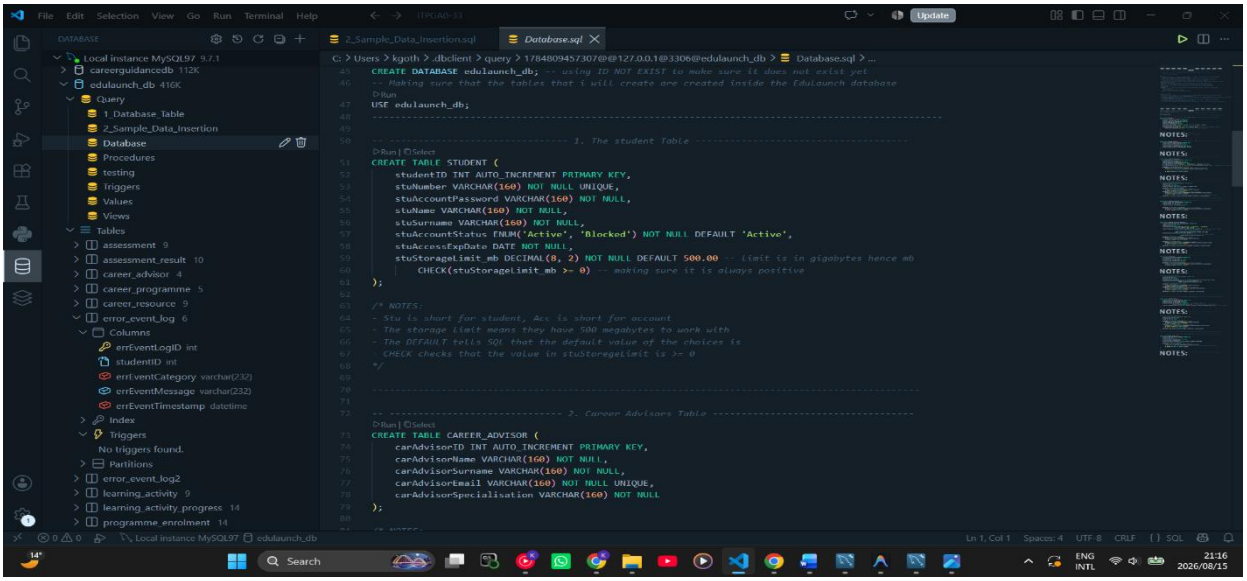
---

3. **Creating Index idx_Student_ProgrammeEnrolment**
   - This index is created on the programme_enrolment table, on the studentID and carProgrammeID columns together.
   - This speeds up checking whether a student is already enrolled into a career programme and counting how many students are currently enrolled into a career programme, which is exactly what spEnrolStudent does every time it is called, when it checks v_alreadyEnrolled and v_currentlyEnrolled. It also speeds up the vwPopularCareerProgrammes view, which joins and counts enrolments per career programme.
   - This screenshot shows how the index idx_Student_ProgrammeEnrolment works using explain:
   -



==========

SECTION D: Rough Work



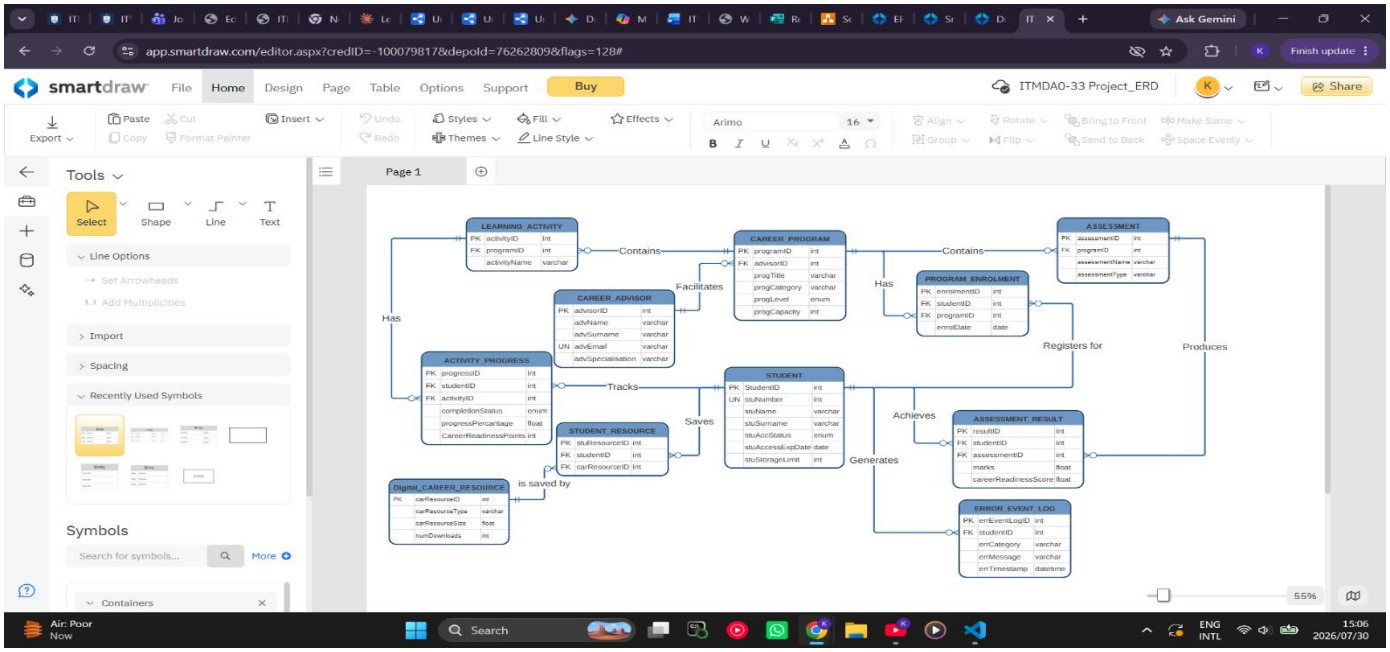
```sql
CREATE DATABASE edulaunch_db; -- using IF NOT EXIST to make sure it does not exist yet
-- Making sure that the tables that i will create are created inside the Edulaunch database
USE edulaunch_db;

-- ------------------------------ 1. The student Table ------------------------------
CREATE TABLE STUDENT (
    studentID INT AUTO_INCREMENT PRIMARY KEY,
    stuNumber VARCHAR(160) NOT NULL UNIQUE,
    stuAccountPassword VARCHAR(160) NOT NULL,
    stuName VARCHAR(160) NOT NULL,
    stuSurname VARCHAR(160) NOT NULL,
    stuAccountStatus ENUM('Active', 'Blocked') NOT NULL DEFAULT 'Active',
    stuAccessExpDate DATE NOT NULL,
    stuStorageLimit_mb DECIMAL(8, 2) NOT NULL DEFAULT 500.00 -- limit is in gigabytes hence mb
        CHECK(stuStorageLimit_mb >= 0) -- making sure it is always positive
);

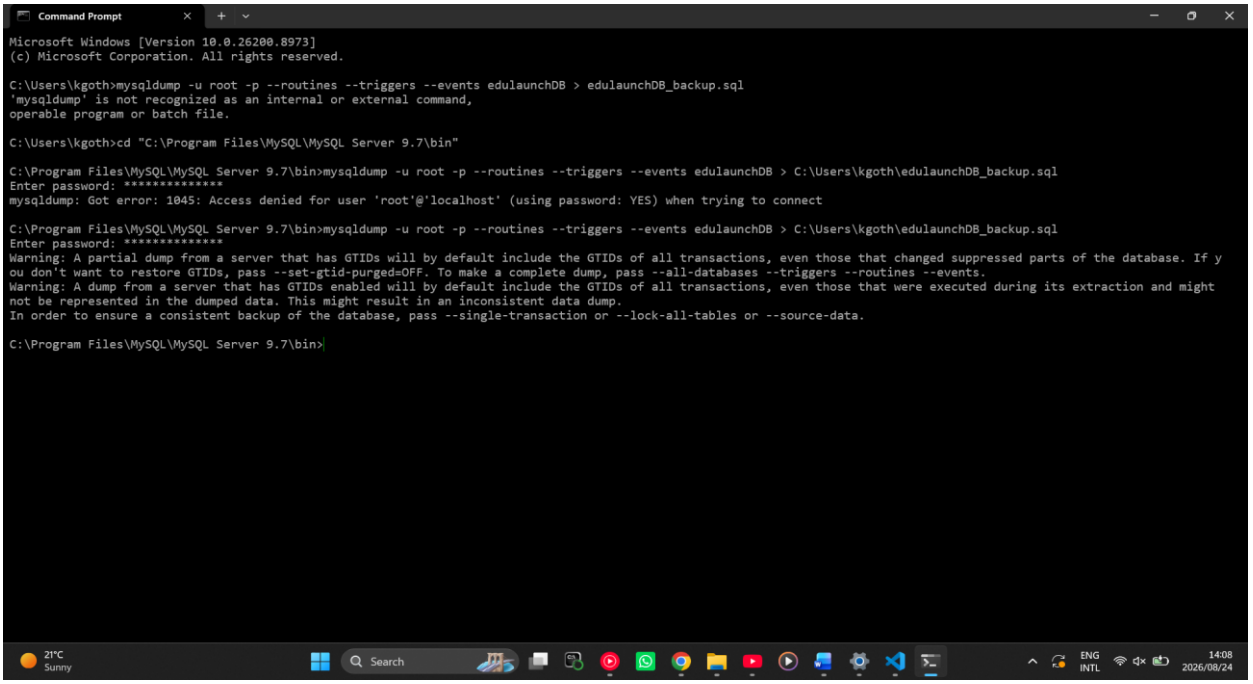
/* NOTES:
- Stu is short for student, Acc is short for account
- The storage limit means they have 500 megabytes to work with
- The DEFAULT tells SQL that the default value of the choices is
  CHECK checks that the value in stuStorageLimit is >= 0
*/

-- ------------------------------ 2. Career Advisors Table ------------------------------
CREATE TABLE CAREER_ADVISOR (
    carAdvisorID INT AUTO_INCREMENT PRIMARY KEY,
    carAdvisorName VARCHAR(160) NOT NULL,
    carAdvisorSurname VARCHAR(160) NOT NULL,
    carAdvisorEmail VARCHAR(160) NOT NULL UNIQUE,
    carAdvisorSpecialisation VARCHAR(160) NOT NULL
);
```

- With VARCHAR size is used to have 160 but decreased in to mostly 65 since miss morgan told me that everything costs memory, so I had to optimise my code.
- My code had notes underneath each code block, but I moved this to user guideline because I wanted my code to look clean and clear



- This is my first ERD, I spelled PROGRAM instead of PROGRAMME, made so many little mistakes so I had to start again.



- This is me creating the SQL DUMP (backup of my database)

SECTION D: Refences

1. **Lucidchart. n.d.** Create database diagrams as a visual starting point for database design. lucidchart.com [Online] Available at: https://www.lucidchart.com/blog/create-database-diagrams-as-a-visual-starting-point-for-database-design [Accessed 12 August 2026].
2. **W3Schools. n.d.** SQL Constraints. [Online] Available at: https://www.w3schools.com/sql/sql_constraints.asp [Accessed 18 August 2026].
3. Eduvos. 2026. Database Design Concepts (2026) (ITDCA0-22 (2026)). myLMS. [Online] Available at: https://mylms.vossie.net/course/view.php?id=37633 [Accessed 13 August 2025].
4. Eduvos. 2026. Database Management (2026) (ITMDA0-33 (2026)). myLMS. [Online] Available at: https://mylms.vossie.net/cource/view.php?id=41868 [Accessed 16 August 2025].

AI Evidence Section

[AI Code 1 – 4 may necessitate evidence submission as specified in your assessment instruction, this section is reserved for evidences where relevant, please delete if not applicable.]

AI Disclosure Appendix

tool used	rpose / how it was used	w I verified or adapted the output
. ChatGPT 4o	. Brainstormed initial argument outline	. Cross-checked with three peer-reviewed sources; fully rewritten in own words
. Claude Opus 4.7	. Brainstormed initial argument outline	. Cross-checked with three peer-reviewed sources; fully rewritten in own words
. Grok 4	. Brainstormed initial argument outline	. Cross-checked with three peer-reviewed sources; fully rewritten in own words

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance with institutional policy.

dent Signature(s)	te
X  Kgothatso Masha Learner	
